03 · Interrupts & Timer

Mikrocomputertechnik 3 · Zeit und Ereignisse im RISC-V-System

Prof. Dr.-Ing. Daniel Klünder

Reicht ein schnellerer Prozessor?

Einstieg · Das Messgerät und das Lehrmodell

Ein Messgerät rechnet gerade 4 ms lang. Neue Sensordaten sollen innerhalb von $100\ \mu$s aus dem Gerät gesichert werden.

- Würde ein doppelt so schneller Prozessor das Problem lösen?
- Woher weiß das Programm, dass Daten vorliegen?
- Was muss die Hardware bis zur Bearbeitung festhalten?

Zwei Minuten zu zweit: Vorschlag und fehlende Annahme nennen.

Notizen

Ein Mikrocontroller wertet gerade 4 ms lang alte Messdaten aus. Währenddessen meldet sein ADC einen neuen Wert, der nach $100\ \mu$s überschrieben wird. Wie kommt dieser Wert rechtzeitig in den Speicher? ADC bedeutet Analog-to-Digital Converter, Analog-Digital-Umsetzer: Ein analoges Eingangssignal wird in einen digitalen Messwert überführt. Die Zahlen beschreiben ein Lehrszenario, keine gemessene Plattform. Die Frist endet beim Sichern des Werts aus dem Gerät; seine vollständige Auswertung darf später erfolgen. Aus den bisherigen Einheiten sind Prozessor, Geräteadressen und unterschiedlich schnelle Speicherzugriffe bekannt. Eine naheliegende Vermutung lautet deshalb, ein schnellerer Prozessor oder Cache werde das Problem lösen. Bei idealer Verdopplung der Rechengeschwindigkeit dauert die Auswertung jedoch noch immer 2 ms. Erfolgt die nächste Geräteabfrage erst danach, kann die Wartezeit die zulässigen $100\ \mu$s weiterhin weit überschreiten. Auch ein schneller Cache legt nicht fest, an welcher Stelle des Programms eine Abfrage stattfindet.

Andere Vorschläge setzen beim Ablauf an: Die Rechnung könnte regelmäßig nachfragen, sie könnte durch ein Ereignis unterbrochen werden, oder zusätzliche Hardware könnte den Wert übernehmen. Jede dieser Ideen benötigt Voraussetzungen. Bleiben Bereitschaft und Messwert bis zur Bearbeitung erhalten? Kann die laufende Rechnung unterbrochen werden? Was geschieht, wenn vor der Übernahme bereits ein zweiter Wert eintrifft? Solche Annahmen entscheiden darüber, ob eine Lösung nur plausibel klingt oder eine zeitliche Zusage tatsächlich trägt.

Die kurze Partnerdiskussion verlangt zunächst einen begründeten Vorschlag und eine fehlende Annahme. Die folgenden Abschnitte entwickeln daraus ein prüfbares Ereignis- und Zeitmodell. Bei der abschließenden Rückkehr zum Messgerät wird dieselbe Frage mit getrennten Zeiten für Datensicherung und Auswertung beantwortet.

Folie 2 / 56

Was sollen Sie entscheiden können?

Einstieg · Das Messgerät und das Lehrmodell

- Ereignisbehandlung begründet auswählen
- Trap- und Gerätepfad vollständig erklären
- Timertermine trotz Verspätung planen
- Reaktionszeit und Datenübergabe prüfen

Lernphase	Zweck
Vorlesung	Zusammenhänge begründen
Übung	Neue Fälle auf Papier lösen
Quiz	Fehlvorstellungen erkennen

Notizen

Ein tragfähiger Entwurf für Zeit und Ereignisse verbindet vier Entscheidungen: Wie erfährt Software von neuer Arbeit? Wie gelangt sie vollständig in die zuständige Behandlung und wieder zurück? Wie bleiben Timertermine trotz Verspätungen sinnvoll geplant? Und wann ist die geforderte Wirkung tatsächlich erreicht?

Polling bezeichnet das Abfragen eines Gerätezustands durch Software. Ein Interrupt ist eine asynchrone Unterbrechungsanforderung. Trap heißt der Eintritt in die Behandlung eines Interrupts oder einer synchronen Ausnahme. Zunächst genügen diese vorläufigen Bedeutungen; im Abschnitt über den Trap-Eintritt werden Ursachen, Freigaben und gesicherter Zustand genauer unterschieden. Entscheidend ist jeweils, einen Ablauf mit seinen Voraussetzungen zu erklären und Fehler darin zu erkennen.

Vorlesung, Übung und Quiz erfüllen dafür verschiedene Aufgaben. Die Vorlesung begründet das gemeinsame Zustands- und Zeitmodell. Die Übung verändert Geräte, Zahlen und Ablaufspuren und fordert eine eigene Übertragung auf neue Fälle. Das Quiz macht vor allem typische Verwechslungen sichtbar: wartende Anforderung gegenüber Freigabe, Controllerübernahme gegenüber Gerätequittierung sowie mittlere Auslastung gegenüber Reaktionsfrist. Ein richtiges Kreuz wird erst durch die passende Begründung aussagekräftig.

Diese Verbindung von Architektur, C-Programmierung und Systementwurf bildet den Kern der Einheit im fünften Semester. Eine vollständige Registerkarte auswendig zu lernen ist dafür nicht erforderlich. Die optionalen Vertiefungen nach dem Abschluss erläutern ausgewählte Implementierungsdetails; Pflichtübung und Quiz setzen sie nicht voraus.

Vorkenntnisse und Lehrmodell

Einstieg · Das Messgerät und das Lehrmodell

Was Sie mitbringen

Geräteadressen, Speicherzugriffe und Cache-Kopien (VL 1–2)

C-Zeiger, Bitoperationen, einfache Schleifen

STM32-Timer mit Prescaler, ARR und CNT aus dem Labor

Vereinfachungen dieser Vorlesung (neu)

RV32: RISC-V mit 32-Bit-Registern

Ein Hart: genau ein Befehlsstrom, vereinfacht ein Prozessorkern

Machine Mode: höchste Privilegstufe mit vollem Zugriff auf Prozessor und Geräte

Keine Verschachtelung: Eine laufende Interruptbehandlung wird nicht von weiteren Interrupts unterbrochen

Geräteadressen und Timerfrequenzen sind plattformabhängig.

Notizen

Das Lehrmodell knüpft an zwei Arten von Vorwissen an. Geräteadressen, Speicherzugriffe und Cache-Kopien stammen aus den Vorlesungen 1–2. Aus der C-Programmierung werden Zeiger als gespeicherte Adressen, Bitoperationen zum Prüfen oder Ändern einzelner Bits und einfache Schleifen benötigt. Gerätezugriffe folgen dem Prinzip Memory-Mapped Input/Output, also Ein- und Ausgabe über Geräteadressen. Ein Gerätestatus kann beim Lesen einen Seiteneffekt besitzen; eine Cache-Kopie ersetzt keine Benachrichtigung über ein neues Ereignis.

Aus dem Labor sind außerdem STM32-Timer mit Prescaler, dem Vorteiler, ARR, Auto-Reload Register, und CNT, Counter, bekannt. ARR legt im betrachteten Aufwärtszählbetrieb die Periodengrenze fest; CNT enthält den laufenden Zählwert. Dieses Wissen hilft später, eine in Hardware wiederholte Periode von einem einzeln programmierten Vergleichstermin zu unterscheiden. Der frei laufende Output-Compare-Betrieb wird dann als Brücke erklärt und nicht als bereits beherrschte Voraussetzung verlangt.

RV32, Hart und Machine Mode bestimmen dagegen neue Modellannahmen. RV32 bezeichnet die RISC-V-Variante mit 32 Bit breiten allgemeinen Integerregistern. Daraus folgt nicht, dass jedes Geräte- oder Timerregister ebenfalls nur 32 Bit breit ist. Ein Hart ist ein Hardware-Ausführungskontext mit einem eigenen Befehlsstrom. Für unser Ein-Hart-Modell genügt die Vorstellung eines einzelnen Prozessorkerns. Allgemein kann ein physischer Kern mehrere solche Ausführungskontexte besitzen. Hauptprogramm und Unterbrechungsbehandlung laufen hier nacheinander auf demselben Hart.

Machine Mode ist die höchste RISC-V-Privilegstufe. Hauptprogramm und Behandlung haben im Lehrmodell dort vollen Zugriff auf die benötigten Prozessorregister und Geräte. Andere Privilegstufen, zusätzliche Zugriffsbeschränkungen und die Weiterleitung einer Behandlung an andere Privilegstufen bleiben ausgeblendet. Keine Verschachtelung bedeutet, dass eine laufende Interruptbehandlung nicht von einem weiteren der betrachteten maskierbaren Interrupts unterbrochen wird. Neue Anforderungen dürfen trotzdem entstehen und warten. Synchrone Exceptions, Ausnahmen durch den gerade ausgeführten Befehl, sind damit nicht ausgeschlossen.

Konkrete Geräteadressen und die Frequenz der Timerzeitquelle liefert die jeweilige Plattform, nicht allein der Name RISC-V. Alle Zahlenbeispiele sind Lehrannahmen. Eine reale Firmware benötigt passende

Gerätezugriffe und eine Plattformbeschreibung; ihre Ausführung wird hier nicht vorausgesetzt. Die Vereinfachungen halten den Ereignis- und Zeitpfad überschaubar, ohne mehrere Kerne und verschachtelte Behandlungen gleichzeitig erklären zu müssen.

Folie 4 / 56

Vier Schulstunden: Ablauf

Einstieg · Das Messgerät und das Lehrmodell

Stunde	Gemeinsame Erarbeitung	Arbeitsphase
1	Ereignisse und Trap-Eintritt: 30 min	Aufgabe 1: 15 min
2	Ursachenauswahl und Gerätepfad: 30 min	Aufgaben 2–3: 15 min
3	Zeitquelle und Sollraster: 30 min	Aufgabe 4: 15 min
4	Reaktionszeit und Datenübergabe: 20 min	Aufgaben 5–6: 10 min; Quiz: 15 min

Die vier Vertiefungsfolien nach dem Abschluss gehören zur Selbstlektüre.

Notizen

Vier Schulstunden ergeben $4 \cdot 45 = 180$ Unterrichtsminuten; Pausen kommen hinzu. Davon sind 110 Minuten für Erklärung, Einstiegsdiskussion, kurze Denkfragen und gemeinsame Auswertung vorgesehen, 55 Minuten für angeleitete Aufgaben und 15 Minuten für das Quiz. Die Unterrichtsstunden bündeln zunächst Ereignisse und Trap-Eintritt, danach Ursachenauswahl und Gerätepfad, anschließend Zeitquelle und Sollraster und zuletzt Reaktionszeit und Datenübergabe.
Die angeleitete Arbeitszeit reicht für ausgewählte Teilaufgaben und den Vergleich der Lösungswege. Für eine vollständige selbstständige Bearbeitung nennt das Übungsblatt etwa 75 bis 90 Minuten. Die sechs Pflichtaufgaben beginnen direkt mit neuen Situationen; offene Teilaufgaben können nach der gemeinsamen Besprechung fertiggestellt werden. Besonders bei Entwurfsfragen zählt die nachvollziehbare Begründung stärker als eine möglichst gleichmäßige Minutenverteilung über die Folien.
Die Abschnittsmarkierungen machen innerhalb dieser Zeitplanung den fachlichen Standort sichtbar. Sie trennen neue Fragen voneinander, die inhaltlich aufeinander aufbauen: Ereignisse müssen erfasst sein, bevor eine Benachrichtigung nützt; ein korrekter Eintritt muss eingerichtet sein, bevor eine Gerätebehandlung sicher laufen kann. Die optionalen Vertiefungen nach dem Abschluss sind Nachschlagematerial zu Gerätezugriffen, Verschachtelung und Controllerdetails. Sie benötigen keine zusätzlich vorausgesetzte Unterrichtszeit und sind keine Voraussetzung für Pflichtübung oder Quiz.

Folie 5 / 56

1 · Ereignisse erfassen

Was bleibt von einem Ereignis, bis die Software reagiert?
Erfassen → Benachrichtigen → Bearbeiten
Gespeicherter Zustand, Polling und Interrupt sind getrennte Entscheidungen.

Notizen

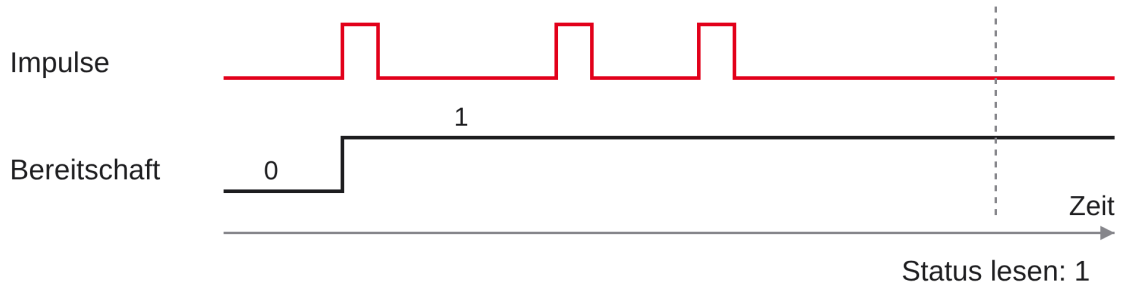
Das Messgerät kann nur auf ein Ereignis reagieren, von dem im richtigen Moment noch eine verwertbare Information vorhanden ist. Bevor Prozessorregister und Unterbrechungsbehandlung betrachtet werden, ist deshalb der Weg vom flüchtigen Gerätesignal zum gespeicherten Zustand zu klären. Eine kurze Meldung kann verschwinden, während die Software mit etwas anderem beschäftigt ist. Ein gespeichertes Bereitschaftsbit bewahrt dagegen einen Hinweis, aber noch keine Anzahl oder Folge von Messwerten.
Dieser erste Abschnitt entwickelt die drei Aufgaben Erfassen, Benachrichtigen und Bearbeiten. Danach lassen sich Polling und Interrupt sachlich vergleichen: Wie erfährt die Software von einem wartenden Zustand, wie groß kann ihre Wartezeit werden, und welche Daten müssen bis dahin erhalten bleiben? Die Kriterien hängen unmittelbar mit der hundert Mikrosekunden langen Sicherungsfrist des Messgeräts zusammen.
Die Reihenfolge schützt vor einem häufigen Entwurfsfehler. Ein anderer Benachrichtigungsweg kann weder ein nicht erfasstes Ereignis wiederherstellen noch fehlenden Speicher ersetzen. Erst wenn der Informationsbedarf und die zeitlichen Grenzen bekannt sind, lohnt sich die Frage, wie der Prozessor die laufende Rechnung für eine Behandlung verlassen kann.

Folie 6 / 56

Drei Schritte bis zur Reaktion

1 / 5 · Ereignisse

Ereignisse, gespeicherter Zustand und spätere Beobachtung



Das Bit erhält einen Hinweis; die Zahl der Impulse bleibt unbekannt.

- Erfassen: Das Ereignis muss einen Zustand hinterlassen
- Benachrichtigen: Software muss davon erfahren
- Bearbeiten: Daten sichern und Ursache behandeln

Notizen

Damit ein Ereignis eine Reaktion auslöst, müssen drei getrennte Aufgaben gelingen: Erfassen, Benachrichtigen und Bearbeiten. Jede kann für sich scheitern. Ein schneller Benachrichtigungsweg allein genügt deshalb nicht für eine zuverlässige Reaktion.
In der oberen roten Bahn liegen drei kurze Geräteimpulse auf einer gemeinsamen Zeitachse. Ein Sensor könnte damit jeweils „neuer Wert da“ melden. Ein solcher Impuls ist flüchtig: Nach seinem Ende bleibt ohne zusätzliche Speicherung keine Spur. Die Achse trägt keine konkrete Zeiteinheit, weil zunächst die Reihenfolge zählt. Die mittlere Bahn zeigt ein Bereitschaftsbit. Als technische Vorstellung eignet sich ein Flipflop, ein Schaltungselement zum Speichern eines Bits. Der erste Impuls setzt den Zustand von null auf eins; der zweite und dritte ändern ihn nicht mehr. Bis zum Rücksetzen bleibt die Eins erhalten.
Die gestrichelte senkrechte Linie kennzeichnet die spätere Statusbeobachtung durch Software. Das Statusregister enthält hier den lesbaren Gerätezustand. Eine gelesene Eins bedeutet „mindestens ein Ereignis ist eingetroffen“. Aus ihr lässt sich nicht erkennen, dass es drei waren. Ein anschließendes Rücksetzen ist in der Zeichnung nicht dargestellt; es gehört im einfachen Bitmodell zur späteren Behandlung.
Erfassen heißt somit, einen bis zur Bearbeitung erhaltenen Zustand zu hinterlassen. Ohne ihn kann ein Impuls vollständig zwischen zwei Softwarebeobachtungen liegen und verloren gehen. Benachrichtigen heißt, dass Software von diesem Zustand erfährt. Beim Polling schaut sie selbst nach. Beim Interrupt fordert Hardware aufgrund des Zustands eine Unterbrechung der laufenden Ausführung an; der tatsächliche Eintritt benötigt zusätzlich passende Freigaben. Bearbeiten heißt schließlich, die benötigten Daten rechtzeitig zu sichern und die Ursache nach Gerätevertrag zu behandeln. Nicht jedes Statusbit wird schon durch Lesen gelöscht.
Wer Polling durch Interrupts ersetzt, verändert zunächst die Benachrichtigung. Ein nicht erfasstes Ereignis wird dadurch nicht nachträglich rekonstruierbar. Umgekehrt sind bei gesetztem Bereitschaftsbit die Daten noch nicht automatisch übernommen oder ausgewertet. Wenn alle drei Ereignisse unterschieden werden müssen, ist mindestens ein passender Zähler nötig. Werden ihre einzelnen Messwerte benötigt, müssen auch die Werte in einem ausreichend großen Datenpuffer erhalten bleiben. Ein Zähler enthält diese Daten nicht.
Ein Briefkasten mit Fähnchen veranschaulicht die Trennung: Das Fähnchen geht beim ersten Brief hoch und bleibt unabhängig von weiteren Briefen oben. Es meldet Post, aber nicht deren Menge. Die aufbewahrten Briefe entsprechen zusätzlich gespeicherten Daten. Vorbeischauen ist Polling; Briefe entnehmen und Fähnchen zurücksetzen ist Bearbeitung. Für das Messgerät bedeutet dies: Mindestens die Bereitschaft muss bis zur Beobachtung erhalten bleiben. Wenn jeder Messwert zählt, müssen zusätzlich die Werte selbst vor dem Überschreiben geschützt werden.

Folie 7 / 56

Polling: selbst nachfragen

1 / 5 · Ereignisse

- Polling: Gerätezustand per Software prüfen
- Busy Waiting: wiederholt prüfen und dabei warten
- Periodisches Polling: zwischen Abfragen andere Arbeit
- Kurze ungespeicherte Ereignisse können zwischen Abfragen verschwinden

Notizen

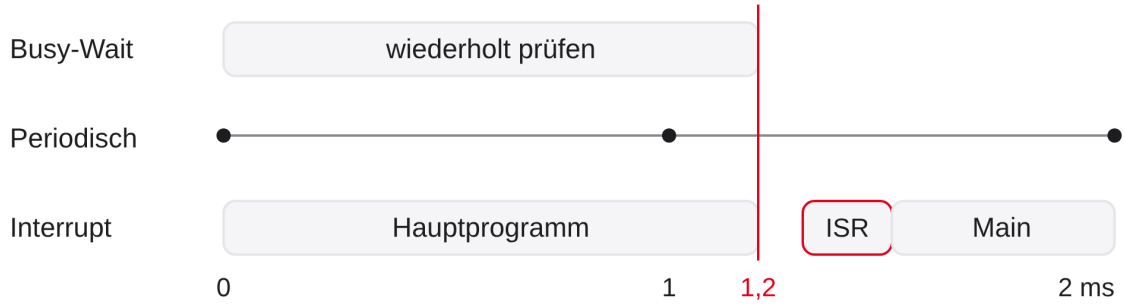
Eine bereits gespeicherte Bereitschaft kann Software durch Polling erkennen. Polling bedeutet Abfragen: Beispielsweise wird ein Gerätestatusregister gelesen und anschließend geprüft, ob Daten bereitliegen. Busy Waiting, beschäftigtes Warten, wiederholt diese Prüfung in einer engen Schleife ohne andere nützliche Arbeit. Periodisches Polling fragt dagegen nur zu bestimmten Zeitpunkten nach und lässt dazwischen Zeit für weitere Aufgaben.
Der Zugriff benutzt Geräteadressen, besitzt aber nicht automatisch die Eigenschaften gewöhnlichen Arbeitsspeichers. Ein Lesezugriff kann etwa Daten entnehmen oder einen Zustand verändern. Der Gerätevertrag bestimmt, welche Beobachtungen zulässig sind und wie eine Ursache anschließend behandelt wird.
Polling kann bei dauerhaft sichtbarer Bereitschaft und tolerierbarer Verzögerung eine einfache, passende Lösung sein. Ein kurzer Impuls ohne Speicherung im Gerät oder in weiterer Hardware kann hingegen vollständig zwischen zwei Prüfungen liegen. Auch ein Interrupt benötigt eine geeignete Erfassung; er ist zunächst ein anderer Weg der Benachrichtigung. Der Vergleich der Reaktionswege muss deshalb bei gleichen Speicherannahmen ansetzen. Erst dann lässt sich sinnvoll untersuchen, wie schnell die Software einen Hinweis entdeckt und wie viel Prozessorzeit dafür gebunden wird.

Folie 8 / 56

Reaktionswege auf einer Zeitachse

1 / 5 · Ereignisse

Modell: Ereignis 1,2 ms · ISR-Eintritt 1,3 ms · Ende 1,5 ms



- Periodische Abfrage: Modellwartezeit $0{,}8$ ms
- Interrupt: eigene Eintrittslatenz und Behandlungszeit

Notizen

Die gemeinsame Zeitachse von null bis 2 ms erlaubt den Vergleich dreier alternativer Programmentwürfe. Die Bahnen stehen nicht für drei gleichzeitig laufende Prozessoren. Bei $1{,}2$ ms tritt das durch die rote senkrechte Linie markierte Ereignis ein, beispielsweise ein neuer Sensorwert. Für alle drei Wege hält ein Bereitschaftsbit den Ereignishinweis bis zur Bearbeitung fest. Diese Annahme speichert weder automatisch den Messwert noch die Anzahl mehrerer Ereignisse.
Oben fragt der Prozessor durch Busy Waiting ständig den Status ab. Im gezeichneten Modell bemerkt er das Ereignis näherungsweise bei $1{,}2$ ms. Auch diese Schleife benötigt in Wirklichkeit Zeit für einzelne Abfragen; die unmittelbare Entdeckung ist eine Vereinfachung. Die schnelle Reaktion bindet den Prozessor während der Wartephase an die Abfragen. Die 4 ms lange Auswertung des Messgeräts käme in dieser

Zeit nicht voran.
In der mittleren Bahn liegen Statusbeobachtungen nur bei null, 1 und 2 ms. Die Punkte markieren die Prüfungen, die Verbindungslinie die dazwischen für andere Arbeit verfügbare Zeit. Das Ereignis bei $1{,}2$ ms folgt auf die Beobachtung bei 1 ms und wird erst bei 2 ms entdeckt. Seine Wartezeit beträgt $2 - 1{,}2 = 0{,}8$ ms. Für eine zeitliche Zusage reicht dieser Einzelablauf noch nicht: Ein Ereignis unmittelbar nach einer Beobachtung wartet nahezu ein ganzes Abfrageintervall.
In der unteren Bahn rechnet zunächst das Hauptprogramm. Nach dem Ereignis beginnt bei $1{,}3$ ms die rot umrandete ISR, Interrupt Service Routine, Unterbrechungsbehandlung. Der Abstand $1{,}3 - 1{,}2 = 0{,}1$ ms heißt hier Eintrittslatenz. Er ist eine Lehrannahme für den gesamten Weg bis zum Beginn der gezeichneten Behandlung, keine gemessene reine Hardwarelatenz. Die Hardware muss die Anforderung erkennen und den Einsprung auslösen; bis zur eigentlichen Gerätearbeit kann zusätzlich eine Software-Sicherung von Registern nötig sein. Die gesamten $100\ \mu$s lassen sich daher nicht allein mit dem Abschließen des laufenden Befehls erklären.
Die ISR endet bei $1{,}5$ ms und dauert im Diagramm $0{,}2$ ms. Danach setzt „Main“, das Hauptprogramm, fort. Eine zusätzliche Wartezeit wegen gesperrter Interrupts ist hier nicht eingezeichnet. Bei einer anderen Freigabesituation muss auch sie im Reaktionsbudget berücksichtigt werden.
Keiner der drei Wege gewinnt ohne weitere Bedingungen. Busy Waiting entdeckt im Modell schnell, belegt aber den Prozessor. Periodisches Polling lässt andere Arbeit zu und kann das Ereignis später entdecken. Ein Interrupt lenkt die Ausführung bei einer Anforderung in einen anderen Pfad um, besitzt aber ebenfalls Eintritts- und Behandlungsaufwand sowie möglicherweise zusätzliche Blockierung. Die $0{,}8$ ms Polling-Wartezeit, die $0{,}1$ ms Eintrittslatenz und die $0{,}2$ ms ISR-Dauer haben verschiedene Anfangs- und Endpunkte und dürfen nicht gleichgesetzt werden.
Ebenso muss der Endpunkt der Anforderung feststehen. Der ISR-Beginn liegt hier $0{,}1$ ms nach dem Ereignis, ihr Ende $0{,}3$ ms danach. Wann innerhalb der ISR Daten gesichert werden, ist nicht angegeben. Setzt die ISR lediglich eine Arbeitsmarkierung, folgt die Nutzarbeit sogar erst später im Hauptprogramm. Mit den gezeichneten Zahlen ist die Datensicherung des Einstiegsfalls deshalb nicht nachgewiesen: Die ISR beginnt erst beim Ablauf seiner $100\ \mu$s-Frist. Die spätere Rechnung unter „Wann ist die Reaktion fertig?“ verwendet andere ausdrücklich angenommene Grenzen: $40\ \mu$s Blockierung, $5\ \mu$s Eintritt und $8\ \mu$s Sicherung ergeben $53\ \mu$s bis zur Datensicherung; das fertige Ergebnis folgt nach $163\ \mu$s. Auch das bleibt eine Modellrechnung und kein Plattformmesswert.

Folie 9 / 56

Wie oft muss Polling nachfragen?

1 / 5 · Ereignisse
Lehrmodell: gespeicherter Status, alle $250\ \mu$s prüfen. Eine Abfrage kostet $3\ \mu$s Prozessorzeit.

$$W_{\text{max}} = 250\,\mu\text{s}, \qquad U = \frac{3}{250} = 1{,}2\,\%$$

- $100\ \mu$s Erkennungsfrist sind damit nicht garantiert
- Kürzeres Intervall: weniger Wartezeit, mehr Abfrageaufwand
- Die größte tatsächliche Abfragelücke zählt

Notizen

Die beobachteten $0{,}8$ ms Wartezeit des vorherigen Beispiels gelten nur für die gewählte Lage des Ereignisses. Eine Zusage muss den ungünstigsten Fall abdecken. Trifft ein gespeichertes Ereignis unmittelbar nach einer Statusbeobachtung ein, wartet es nahezu eine volle Abfrageperiode. Bezeichnet P den tatsächlichen Abstand zwischen den Beobachtungen, dann ist $W_{\text{max}} = P$ eine obere Grenze der Wartezeit bis zur nächsten Beobachtung. Bei $P = 250\ \mu$s ist bereits die Erkennung innerhalb von $100\ \mu$s nicht garantiert; die anschließende Datensicherung benötigt darüber hinaus Zeit.

Die Abfrageperiode bestimmt zugleich den Preis des Nachfragens. Der angenommene gesamte Prozessoraufwand einer Abfrage beträgt $C = 3\ \mu$s. Eine Prüfung je Periode belegt im Mittel den Anteil $U = C/P = 3/250 = 0{,}012 = 1{,}2$ Prozent der Prozessorzeit. Die übrige Zeit steht im Modell anderer Arbeit zur Verfügung. Wird die Periode bei unverändertem Abfrageaufwand auf $50\ \mu$s verkürzt, steigt der Anteil auf $3/50 = 0{,}06 = 6$ Prozent. Kürzere Wartezeit und geringerer Prozessoraufwand stehen somit gegeneinander. Ein kleiner mittlerer Aufwand beweist keine kurze Reaktion.

Busy Waiting lässt sich als Grenzfall verstehen, in dem zwischen Abfragen keine andere Arbeit mehr liegt. Nähert sich P dem Abfrageaufwand C, nähert sich $U = C/P$ eins, also hundert Prozent. Die Wartezeit wird klein, aber nicht automatisch null. Eine Periode unterhalb des gesamten Abfrageaufwands wäre auf dem einzelnen Prozessor ohne Überlappung nicht ausführbar. Die verwendeten Zahlen beschreiben Lehrannahmen, keine Geräte- oder Compilermessung.

Entscheidend bleibt die größte tatsächliche Abfragelücke. Die Formel setzt voraus, dass Statusbeobachtungen wirklich spätestens nach der angegebenen Periode erfolgen. Enthält das Programm gelegentlich eine 4 ms lange Rechnung ohne Abfrage, ist diese Lücke maßgebend, auch wenn die Hauptschleife sonst meist schnell ist. Eine durchschnittliche Schleifendauer genügt für die Zusage nicht. Genau darin liegt das Problem des Messgeräts.

Eine Polling-Lösung könnte die Rechnung in kurze Abschnitte zerlegen und dazwischen den Status prüfen. Als zusätzliche Lehrannahme seien von der Statusbeobachtung bis zum abgeschlossenen Sichern höchstens $8\ \mu$s nötig; weitere Verzögerungen seien ausgeschlossen. Eine tatsächliche Beobachtungslücke von höchstens $90\ \mu$s ergäbe dann $90 + 8 = 98\ \mu$s bis zur Datensicherung und $2\ \mu$s Reserve gegenüber der Frist. Die $90\ \mu$s sind eine gewählte Budgetgrenze, keine allgemeine Gerätevorgabe. Der Abfrageaufwand von $3\ \mu$s steckt bereits in der Periode zwischen Beobachtungen und wird nicht pauschal ein zweites Mal zur Wartezeit addiert.

Über 4 ms verteilt entspräche $4000/90 \approx 44{,}4$ einer Größenordnung von etwa 45 Beobachtungen. Eine Schleife darf dafür dieselbe Abfragestelle wiederholt benutzen. Der zusätzliche Aufwand verlängert allerdings die ursprünglich 4 ms reine Auswertung, sodass die genaue Zahl von der entstehenden Gesamtdauer abhängt. Außerdem müssen alle Abschnitte auch bei ungünstigen Speicherzugriffen kurz genug bleiben. Eine solche Aufteilung ist möglich, kann aber unhandlich und fehleranfällig werden. Ein Interrupt motiviert einen anderen Ausführungspfad, der keine entsprechend verteilten Abfragen in der Rechnung benötigt. Auch dieser Pfad braucht anschließend ein geprüftes Budget für Blockierung, Eintritt und Datensicherung.

Gespeichert heißt nicht gezählt

1 / 5 · Ereignisse

Vor einer Behandlung treten drei Ereignisse ein.

Hardwarezustand	Was kann Software daraus erkennen?
Ungespeicherter kurzer Impuls	später möglicherweise nichts
Ein Bereitschaftsbit	mindestens ein Ereignis
Ereigniszähler ohne Überlauf	Anzahl der Ereignisse
Ausreichend großer Datenpuffer	gespeicherte einzelne Daten

Welche Information braucht das Messgerät aus dem Einstieg?

Notizen

Drei Ereignisse vor der Bearbeitung können sehr unterschiedliche Spuren hinterlassen. Die Tabelle ordnet die Möglichkeiten nach ihrem Informationsgehalt: Ein ungespeicherter kurzer Impuls hinterlässt später womöglich nichts; ein Bereitschaftsbit erhält „mindestens ein Ereignis“; ein nicht überlaufender Ereigniszähler erhält die Anzahl; ein ausreichend großer Datenpuffer bewahrt zusätzlich die einzelnen Werte. Die Voraussetzungen gehören jeweils zur Aussage: Ein überlaufener Zähler oder ein voller Puffer kann Information verlieren.

Das gespeicherte Objekt muss zur Anforderung passen. Ein Bit besitzt nur zwei Zustände. Nach dem ersten Setzen auf eins ändern weitere Ereignisse diesen Zustand nicht mehr. Die Anzahl drei ist daraus nicht rekonstruierbar. Ein Zähler kann die Ereignisanzahl innerhalb seiner Kapazität erhalten, enthält aber noch keine Messwerte. Ein Datenpuffer benötigt neben ausreichender Größe auch eine korrekt organisierte Übergabe an die bearbeitende Software.

Im Messgerät ist zunächst gefordert, neue Sensordaten rechtzeitig aus dem Gerät zu sichern. Noch offen bleibt, ob jeder aufgenommene Wert benötigt wird oder nur der jeweils neueste. Diese Frage muss vor der Wahl von Flag, Zähler oder Puffer beantwortet werden. Ein Flag, eine einzelne Zustandsmarkierung, passt etwa zu „den aktuellen Zustand erneut auswerten“, nicht zu „alle seitdem aufgenommenen Werte liefern“. Auch ein Pending-Bit, die Markierung einer wartenden Interruptanforderung, ist kein unbegrenzter Ereignisspeicher. Mehrere Ereignisse können vor einer Behandlung zusammenfallen. Die Unterscheidung zwischen Hinweis, Anzahl und Daten betrifft daher Polling und Interrupts gleichermaßen. Sie kehrt später wieder, wenn eine kurze Unterbrechungsbehandlung Arbeit an das Hauptprogramm übergibt.

Welche Strategie passt zum Ereignis?

1 / 5 · Ereignisse

Situation	Zu prüfende Bedingung
Bereitschaft bleibt sichtbar	zulässige Abfrageperiode
Kurzer Impuls	Erfassung oder Speicherung
Sehr viele Ereignisse	Behandlungsaufwand und Puffer
Feste Reaktionsfrist	gesamte Verzögerung

Notizen

Die Wahl des Reaktionswegs beginnt bei den Anforderungen des Ereignisses. Die Tabelle liefert deshalb keine feste Zuordnung von Geräten zu Polling oder Interrupts. Sie bündelt mehrere Prüfungen, die bei derselben Anwendung zugleich nötig sein können. Bleibt Bereitschaft sichtbar, muss die größte Abfragelücke zur Frist passen. Bei einem kurzen Impuls ist vorher zu klären, ob überhaupt eine Erfassung oder Speicherung erfolgt. Viele Ereignisse erfordern eine Betrachtung von Behandlungsaufwand und Pufferkapazität. Eine feste Reaktionsfrist verlangt schließlich die gesamte Verzögerung bis zur geforderten Wirkung.

Die Speicherfrage steht dabei vor der Benachrichtigung. Ein nicht erfasster Impuls kann verschwinden, bevor Software reagiert. Ein einzelnes Pending-Bit bewahrt nur eine wartende Anforderung, keine beliebige Ereignisanzahl. Weder häufigeres Nachfragen noch ein anderer Einsprungmechanismus kann fehlende Information nachträglich ergänzen.

Mit steigender Ereignisrate wird außerdem der wiederholte Behandlungsaufwand relevant. Eine ISR, Interrupt Service Routine oder Unterbrechungsbehandlung, benötigt Ein- und Austritt sowie die jeweilige Gerätearbeit. Für das folgende Lehrbeispiel nehmen wir $5\,\mu$s Gesamtaufwand je vollständiger ISR an, einschließlich Ein- und Austritt. Dieselbe Annahme wird später unter „Kurze ISR: Aufwand berechnen“ rechnerisch eingeordnet. Sie ist weder ein Messwert noch eine isolierte Angabe für den Hardwareeinsprung.

Als Beispiel dient eine UART, Universal Asynchronous Receiver/Transmitter, eine serielle Sende- und Empfangsschnittstelle. Beim kontinuierlichen Empfang im angenommenen Format 8N1 enthält ein Rahmen acht Datenbits, kein Paritätsbit zur Fehlererkennung und ein Stoppbit. Mit dem Startbit sind es zehn Leitungsbits pro Nutzbyte. Start- und Stoppbit kennzeichnen Beginn und Ende des Rahmens. Baud bedeutet Symbole pro Sekunde; bei der hier binären Übertragung entspricht ein Symbol einem Bit. Weitere Übertragungsdetails gehören zur späteren Einheit über serielle Bussysteme.

Bei 115200 Baud kommen $115200/10 = 11520$ Byte je Sekunde an. Löst jedes Byte genau eine vollständige ISR mit dem angenommenen Aufwand aus, ergibt sich

$$U = 11520\,\mathrm{s}^{-1} \cdot 5\,\mu\mathrm{s} = 0{,}0576 = 5{,}76\,\% \approx 6\,\%.$$

Bei einem MBaud, also einer Million Symbole je Sekunde, wären es 100000 Byte je Sekunde und $U = 100000 \cdot 5 \cdot 10^{-6} = 0{,}5 = 50$ Prozent. Die Rechnung setzt das genannte Rahmenformat, lückenlosen Empfang und die konstante angenommene Dauer voraus. Ein anderer Treiber oder Pausen im Datenstrom verändern das Ergebnis.

Ein Gerätepuffer kann mehrere Byte sammeln, bevor eine gemeinsame Benachrichtigung erfolgt. Über-

nimmt eine Behandlung mehrere Byte, verteilen sich feste Ein- und Austrittskosten auf mehr Daten. Die Arbeit für deren Übernahme und Verarbeitung verschwindet dadurch nicht. Ein zusätzlicher Softwarepuffer allein senkt die Interruptzahl nicht, solange jedes Byte weiterhin einen eigenen Interrupt auslöst. Außerdem muss die zusätzliche Sammelwartezeit zur Frist passen und die Pufferkapazität zum Datenaufkommen. Es gibt daher keine allgemeine Ereignisrate, ab der eine bestimmte Technik stets gewinnt.
Eine Reaktionsfrist bezeichnet die späteste zulässige Zeit für eine benannte Wirkung. Auf dem Weg liegen Erfassung, mögliche Wartezeit, Eintritt und Nutzarbeit. Eine kurze mittlere ISR-Dauer begrenzt nicht die Wartezeit eines einzelnen Ereignisses. Das Setzen einer Arbeitsmarkierung ist auch noch keine abgeschlossene Auswertung. Unter „Wann ist die Reaktion fertig?“ werden deshalb $53\ \mu$s bis zur Datensicherung und $163\ \mu$s bis zum fertigen Ergebnis unterschieden.
Beim Messgerät greifen mehrere Tabellenzeilen ineinander. Unter der Annahme eines gespeicherten Bereitschaftsbits bleibt der Hinweis sichtbar; trotzdem ist die Abfrageperiode zu prüfen. Zugleich müssen die Daten innerhalb von $100\ \mu$s gesichert sein. Im Dauerbetrieb können darüber hinaus Ereignisrate, Aufwand und Pufferkapazität begrenzend wirken. Erst wenn die fehlenden Angaben, etwa Übernahmedauer und Ereignisabstand, bekannt sind, lässt sich eine Lösung begründen. Darauf baut nun der Prozessorpfad auf: Wie gelangt die Ausführung von einer wartenden Anforderung in die richtige Behandlung?

Folie 12 / 56

2 · In die Behandlung wechseln

Wie unterbricht der Prozessor und setzt anschließend korrekt fort?
Ereignisse → Trap-Eintritt → Geräte → Timer → Reaktionszeit
Jetzt: Freigaben, gesicherter Zustand, Ursache und erster Einsprung.

Notizen

Bisher ist geklärt, welche Information ein Gerät speichern muss und welche Zeitgrenzen ein Benachrichtigungsweg benötigt. Ein Interrupt soll nun einen anderen Ausführungspfad eröffnen, während das Hauptprogramm rechnet. Dazu muss der Prozessor die Unterbrechung annehmen dürfen, den nötigen Rückkehrzustand erhalten und an einer vorbereiteten Stelle mit der Behandlung beginnen.
Dieser Übergang heißt Trap. Er wird sowohl für asynchrone Interrupts als auch für synchrone Exceptions verwendet, obwohl beide unterschiedliche Ursachen haben. Im zweiten Abschnitt werden diese Ursachen, die Freigaben und die Aufgaben von Hardware und Software voneinander getrennt. Erst ihr Zusammenspiel erklärt, warum ein wartendes Gerät nicht immer sofort behandelt wird und warum ein bloßer Sprung zu einer C-Funktion keine vollständige Unterbrechungsbehandlung darstellt.
Am Ende dieses Abschnitts ist der Weg bis zum ersten Handlerbefehl bekannt. Noch offen bleibt dann die Auswahl eines konkreten externen Geräts: Die Prozessorursache kann die Kategorie „extern“ melden, ohne bereits die Geräte-ID zu nennen. Diese zweite Auswahl übernimmt anschließend der externe Interruptcontroller.

Folie 13 / 56

Interrupt, Exception und Trap

2 / 5 · Trap-Eintritt

- Interrupt: asynchrone Anforderung, etwa vom Timer
- Exception: synchron zum ausgeführten Befehl, etwa ein ungültiger Befehl
- Trap: gemeinsamer Übergang in die Behandlung
- Gleicher Einsprungmechanismus bedeutet nicht gleiche Ursache

Notizen

Nach der Wahl eines Benachrichtigungswegs ist zu klären, welche Ursache den Prozessor aus seiner laufenden Ausführung herausführt. Ein Interrupt entsteht asynchron, also unabhängig vom gerade ausgeführten Befehl. Beispielsweise erreicht eine Hardware-Zeitquelle ihren programmierten Vergleichswert. Eine Exception, eine Ausnahme, entsteht dagegen synchron aus der Befehlsausführung, etwa beim Erkennen eines nicht unterstützten Befehls. Trap bezeichnet den gemeinsamen Übergang in die jeweilige Behandlung.
Interrupt und Exception können denselben Eintrittsmechanismus und dieselben besonderen Register nutzen, verlangen aber nicht dieselbe Fortsetzung. Nach einem Interrupt wird gewöhnlich die unterbrochene Ausführung fortgesetzt. Bei einer Exception muss Software entscheiden, ob und wie die auslösende Situation zu behandeln ist. Eine Rückkehr ohne passende Änderung kann unmittelbar denselben Fehler erneut auslösen.
Der Unterschied lässt sich mit dem Lesen eines Buchs vergleichen. Klingelt währenddessen das Telefon, entspricht dies dem Interrupt. Ein unbekanntes Wort im gerade gelesenen Satz entspricht einer Exception. In beiden Fällen wird ein Lesezeichen eingelegt und die Tätigkeit unterbrochen: Das ist der Trap. Nach dem Telefonat kann das Lesen weitergehen. Das unbekannte Wort muss dagegen erst nachgeschlagen oder auf andere Weise behandelt werden, sonst entsteht beim Weiterlesen dieselbe Schwierigkeit erneut.
Interruptfreigaben sperren im folgenden Modell die betrachteten maskierbaren Interrupts. Sie verhindern nicht automatisch synchrone Exceptions durch einen ungültigen Befehl oder Speicherzugriff. „Interrupts aus“ bedeutet deshalb nicht, dass überhaupt kein Trap auftreten kann. Diese Grenze erklärt die getrennte Ursachenkennzeichnung und wird auch bei einer bereits laufenden Unterbrechungsbehandlung wichtig.

Folie 14 / 56

Pending ist nicht Freigabe

2 / 5 · Trap-Eintritt
Im betrachteten Machine Mode:

$$\text{Interruptnahme} : \quad \text{Pending} \wedge \text{Quellenfreigabe} \wedge \text{MIE}$$

- `mip`: Machine Interrupt Pending
- `mie`: Machine Interrupt Enable, Kategorienfreigaben
- `mstatus.MIE`: globale Freigabe der betrachteten M-Interrupts

CSR: Control and Status Register, Steuer- und Statusregister. `mip`, `mie` und `mstatus` sind CSRs im Prozessor; `MIE` ist ein Bit in `mstatus`.

```
csrsi mstatus, 8    # MIE setzen, andere Bits erhalten
```

Notizen

Eine wartende Anforderung reicht noch nicht für den Eintritt in ihre Behandlung. Im vereinfachten Modell läuft ein Hart bereits im Machine Mode, und die Behandlung wird nicht an eine andere Privilegstufe weitergeleitet. Hier müssen Pending-Zustand, Kategorienfreigabe und globale Freigabe gemeinsam vorliegen. Das logische Und $\wedge$ in der Formel drückt genau diese Gleichzeitigkeit aus.
Die zugehörigen Zustände liegen in CSRs, Control and Status Registers, also Steuer- und Statusregistern des Prozessors. CSRs werden durch eigens dafür vorgesehene Prozessorbefehle angesprochen und nicht durch gewöhnliche Zugriffe auf Geräteadressen. `mip`, Machine Interrupt Pending, zeigt wartende Anforderungen der Interrupt-Kategorien. `mie`, Machine Interrupt Enable, enthält deren einzelne Freigaben. `mstatus`, Machine Status, enthält unter anderem das MIE-Bit, Machine Interrupt Enable, als globale Freigabe der betrachteten Maschinen-Interrupts. Der Registername `mie` und das einzelne Bit MIE in `mstatus` bezeichnen verschiedene Dinge; `mstatus.MIE` schreibt die Zuordnung ausdrücklich hin.
Wartet ein Timer bei deaktivierter Timerkategorie, wird der entsprechende Interrupt nicht genommen. Ein freigegebener Timer ohne Pending-Zustand führt ebenfalls nicht zum Eintritt. Ist MIE null, bleiben die betrachteten maskierbaren Maschinen-Interrupts trotz Pending und Kategorienfreigabe zurückgestellt. Das Sperren löscht ihre Ursache nicht automatisch. Eine erhaltene Anforderung kann nach Wiederfreigabe wirksam werden. Ein einzelnes Pending-Bit speichert dabei weiterhin keine Anzahl aller während der Sperre aufgetretenen Ereignisse. Synchrone Exceptions sind durch diese Freigabeformel nicht beschrieben.
Der Lehrbefehl `csrsi mstatus, 8` veranschaulicht einen gezielten Registerzugriff. `csrsi` bedeutet CSR Set Immediate, Setzen mit einer unmittelbar im Befehl enthaltenen Bitmaske. MIE liegt architektonisch fest an Bitposition drei, von null gezählt; die Maske ist $2^3 = 8$. Bitweises Oder setzt dieses Bit und erhält die anderen Registerbits. Der Befehl ersetzt somit nicht den gesamten Registerinhalt durch acht. Entsprechend löscht `csrci mstatus, 8`, CSR Clear Immediate, das MIE-Bit und erhält die übrigen Felder.
Das Beispiel setzt die CSR-Befehle und ausreichende Berechtigung im Machine Mode voraus; es ist keine hier ausgeführte Firmware. Vor der globalen Freigabe müssen Stack, Trap-Eintritt und Gerätebehandlung vorbereitet sein, denn eine bereits wartende, einzeln freigegebene Anforderung kann danach sofort zum Trap führen. Die Initialisierungsfolge im Timerabschnitt greift diese Abhängigkeit wieder auf. Die

Befehlssemantik ist in der RISC-V-Spezifikation der CSR-Befehle beschrieben.

Folie 15 / 56

Welche Anforderung wird angenommen?

2 / 5 · Trap-Eintritt

Ein Timer hat eine wartende Anforderung. Im Machine-Mode-Modell:

Timerfreigabe MTIE	Globale Freigabe MIE	Timer-Trap möglich?
0	0	?
0	1	?
1	0	?
1	1	?

Was passiert beim Sperren mit Zeitquelle und Ursache?

Notizen

Die vier Kombinationen der Tabelle prüfen das gemeinsame Wirken von zwei Freigaben bei bereits wartender Timeranforderung. MTIE heißt Machine Timer Interrupt Enable und bezeichnet die Kategorienfreigabe des Timers. MIE, Machine Interrupt Enable, ist hier das globale Freigabebit in `mstatus`. Ein Timer-Trap, der Eintritt in seine Behandlung, ist unter den weiterhin geltenden Annahmen nur in der letzten Zeile möglich: MTIE und MIE müssen beide eins sein. Das Programm läuft bereits im Machine Mode; eine Weiterleitung an andere Privilegstufen findet nicht statt.

In jeder anderen Zeile fehlt mindestens eine Freigabe. „Möglich“ ist jedoch keine Zusage über die Eintrittsdauer. Die Zeitquelle läuft auch bei gesperrter Freigabe weiter. Ebenso beseitigt das Ändern einer Freigabe nicht automatisch die Timerursache. Besteht sie nach dem erneuten Einschalten noch, kann sie weiter wirksam werden.

Damit lassen sich zwei verbreitete Fehlvorstellungen ausschließen. Das Abschalten von Interrupts hält nicht die Hardwarezeit an. Ebenso folgt aus dem Wiedereinschalten nicht, dass für jede zwischenzeitlich verstrichene Periode eine einzelne Behandlung nachgeliefert wird. Eine wartende Anforderung ist zunächst ein Zustand, keine Ereignisliste. Diese Unterscheidung wird bei der Planung versäumter Timertermine erneut benötigt.

Kontext und spezielle Register

2 / 5 · Trap-Eintritt

Zustand	Bedeutung
Allgemeine Register	laufende Werte und Adressen
`mepc`	gespeicherte Fortsetzungsadresse
`mcause`	Trap-Ursache
`mstatus`	Status und Interruptfreigaben
`mip`	wartende Interruptanforderungen
`mie`	Freigaben der Interrupt-Kategorien

Notizen

Eine Unterbrechungsbehandlung muss die laufende Ausführung später so fortsetzen können, als seien ihre noch benötigten Werte erhalten geblieben. Kontext bezeichnet diesen zur Fortsetzung erforderlichen Zustand: allgemeine Register mit Zwischenwerten und Adressen sowie die relevanten besonderen Statusinformationen. Ein Sprung an die richtige Stelle allein bewahrt diesen Zustand noch nicht.
Die besonderen Prozessorregister sind CSRs, Control and Status Registers. `mepc`, Machine Exception Program Counter, hält die für die Trap-Rückkehr gespeicherte Befehlsadresse. PC bedeutet Program Counter, Befehlsadresszähler. `mcause`, Machine Cause, enthält die Trap-Ursache. `mstatus`, Machine Status, enthält unter anderem die globale Interruptfreigabe MIE. `mip`, Machine Interrupt Pending, zeigt wartende Anforderungen, und `mie`, Machine Interrupt Enable, enthält die Freigaben der Kategorien. Das Präfix `m` verweist auf den Machine Mode, die betrachtete Maschinenprivilegstufe. Auch hier sind das Register `mie` und das MIE-Bit in `mstatus` zu unterscheiden.
Beim Trap aktualisiert Hardware einige dieser speziellen Register. Sämtliche allgemeinen Register werden dadurch aber nicht automatisch auf dem Softwarestack gesichert. Der Stack ist ein Speicherbereich für Aufruf- und Sicherungsrahmen. Verändert die Behandlung Register, in denen das Hauptprogramm noch benötigte Werte hält, muss Software diese Werte passend sichern und wiederherstellen.
Die Zustandsübersicht ist keine pauschale Liste zurückzuschreibender Register. Pending-Bits zeigen beispielsweise aktuelle Anforderungen an und werden nicht einfach durch einen alten Programmzustand ersetzt. Welche Software-Sicherungen tatsächlich erforderlich sind, hängt von den veränderten Registern und vom Behandlungspfad ab. Der folgende Ablauf trennt deshalb ausdrücklich, was Hardware beim Eintritt leistet und was die Software ergänzen muss.

Trap-Eintritt: Hardware und Software

2 / 5 · Trap-Eintritt



Hardware hält Trap-Zustand; Software erhält weitere Register.

- Hardwarezustand und Software-Sicherung ergänzen sich
- `mret` ersetzt keine Wiederherstellung allgemeiner Register

Notizen

Beim Trap-Eintritt greifen Hardware und Software ineinander. Der dargestellte Ablauf liest sich von links nach rechts: Zunächst läuft das Hauptprogramm. Hardware schreibt die Fortsetzungsadresse nach `mepc`, Machine Exception Program Counter, und die Ursache nach `mcause`, Machine Cause. Das bisherige MIE-Bit, Machine Interrupt Enable, wird nach MPIE, Machine Previous Interrupt Enable, kopiert; anschließend wird MIE auf null gesetzt. MPP, Machine Previous Privilege, hält die vorherige Privilegstufe fest. Im Lehrmodell ist diese Stufe bereits der Machine Mode.
Der Machine Mode lässt sich als Hausmeister mit Generalschlüssel vorstellen, der User Mode als Mieter mit Zugang nur zur eigenen Wohnung. Das Bild erklärt höchste gegenüber eingeschränkten Rechten. Die konkreten Grenzen für Speicher- und Gerätezugriffe hängen allerdings von den eingerichteten Schutzmechanismen ab. Hier laufen Hauptprogramm und Unterbrechungsbehandlung beide im Machine Mode; der Trap ist deshalb kein Wechsel aus dem User Mode in den Machine Mode.
Nach den Hardwareaktionen erhält Software im Prolog, dem Eintrittsteil der Behandlung, die benötigten allgemeinen Register in einem Stackrahmen. Erst dann kann der Handler, die Behandlungsroutine, seine Aufgabe ausführen, ohne noch benötigte Werte des Hauptprogramms ungesichert zu zerstören. Im Epilog, dem Austrittsteil, stellt Software diese Sicherungen wieder her. Der Befehl `mret`, Machine Return, nutzt anschließend `mepc` als Rückkehradresse und stellt unter anderem MIE aus MPIE sowie die vorherige Privilegstufe wieder her.
Die Kästen unterscheiden die Hardwareaktionen von den Softwareaufgaben. Ihre Pfeile geben die Reihenfolge an, keine gemessenen Dauern. `mret` lädt nicht von selbst den Softwarestack und ersetzt keine Wiederherstellung allgemeiner Register. Eine fehlende Sicherung kann daher falsche Werte im fortgesetzten Hauptprogramm verursachen, obwohl die Rückkehradresse korrekt ist. Ebenso ist ein gewöhnlicher Funktionsrücksprung kein Ersatz für die Trap-Rückkehr. Nach dem gesicherten Eintritt bleibt nun die passende Behandlung anhand der Ursache auszuwählen.

`mcause` auf RV32 lesen

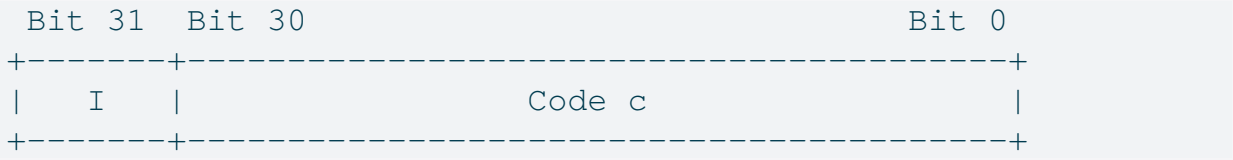
2 / 5 · Trap-Eintritt

$$I = (\text{raw} \gg 31)\ \&\ 1, \qquad c = \text{raw}\ \&\ \texttt{0x7FFFFFFF}$$

Rohwert	Interpretation
`0x80000007`	Interrupt, Kategorie 7
`0x8000000B`	Interrupt, Kategorie 11
`0x00000002`	Exception, Code 2

Notizen

Zur Auswahl der Behandlung liefert `mcause`, Machine Cause, zwei Informationen in einem Wort. Auf RV32 ist das Register 32 Bit breit. Sein höchstes Bit I an Position 31 kennzeichnet einen Interrupt mit eins und eine Exception mit null. Die übrigen 31 Bits enthalten den Ursachencode c. Die Bedeutung des Codes hängt von der ersten Information ab: Interrupt- und Exception-Codes gehören zu unterschiedlichen Namensräumen.



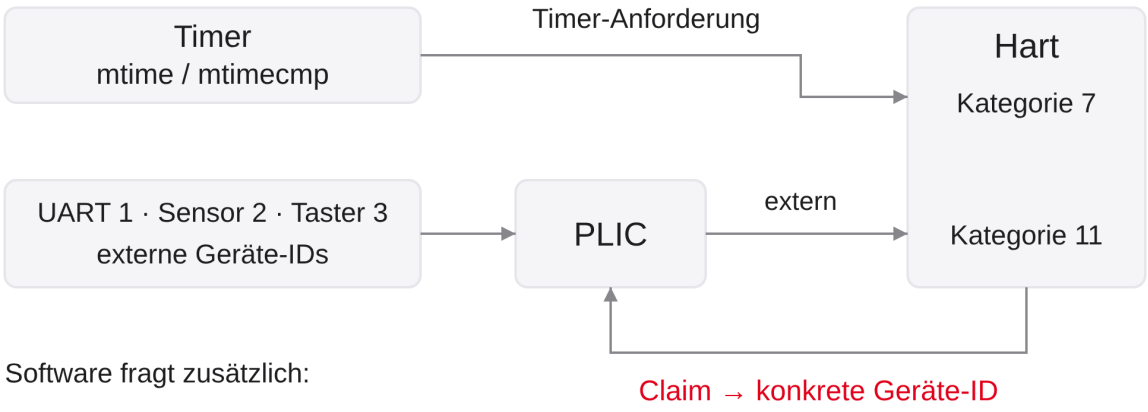
`raw` bezeichnet in den Formeln den vollständigen Rohwert des Registers. Die Rechtsverschiebung $\gg 31$ bringt das höchste Bit an die niedrigste Position; bitweises Und mit eins isoliert daraus I. Die hexadezimale Maske `0x7FFFFFFF` enthält an Bit 31 eine Null und an den übrigen Stellen Einsen. Bitweises Und mit dieser Maske entfernt das Interrupt-Bit und erhält den Code. Das Präfix `0x` kennzeichnet die hexadezimale Schreibweise.

Für `0x80000007` ergibt sich $I = 1$, $c = 7$: Machine Timer Interrupt. `0x8000000B` hat ebenfalls $I = 1$, aber $c = 11$ und bezeichnet einen Machine External Interrupt, die externe Maschinenkategorie. `0x00000002` hat $I = 0$, $c = 2$ und steht für die Exception Illegal Instruction, einen ungültigen Befehl. Die Spezifikation verwendet für das Codefeld auch den Ausdruck Exception Code, wenn das Interrupt-Bit gesetzt ist; die Trennung zwischen Bit und Code bleibt dennoch nötig.

Ein Interrupt ist eine asynchrone Anforderung, eine Exception eine synchrone Ausnahme aus der Befehlsausführung. Wer nur die niedrigen Bits vergleicht, kann deshalb eine Zahl aus dem falschen Namensraum deuten. Eine konkrete externe Geräte-ID liefert `mcause` außerdem noch nicht. Die Kategorie beschreibt zunächst die Art des Trap-Pfads. Welche Gerätequelle hinter der externen Kategorie steht, erfordert einen zusätzlichen Auswahlschritt.

Kategorie ist nicht Geräte-ID

2 / 5 · Trap-Eintritt



- `mcause`: Welche Art von Trap liegt vor?
- PLIC-Claim: Welches externe Gerät wurde übernommen?

Notizen

Bei externen Interrupts erfolgt die Identifikation in zwei Stufen. `mcause`, Machine Cause, nennt zunächst die Art des Interrupts. Erst ein Zugriff auf den PLIC, Platform-Level Interrupt Controller, liefert die konkrete externe Quelle, die zur Bearbeitung übernommen wird. Kategoriencode und Geräte-ID beantworten verschiedene Fragen und dürfen trotz möglicherweise gleicher Zahlen nicht gleichgesetzt werden.
Die Grafik stellt zwei Informationswege zum Hart gegenüber, einem Hardware-Ausführungskontext des Prozessors. Oben führt der lokale Timerpfad mit `mtime`, Machine Time, und `mtimecmp`, Machine Time Compare, ohne PLIC zum Hart. `mtime` ist die Zeitquelle, `mtimecmp` der Vergleichswert. Der Pfad meldet Kategorie sieben, Machine Timer Interrupt. Im klassischen Modell hat jeder Hart seinen zugeordneten Timervergleich; die Zeitquelle selbst kann gemeinsam genutzt werden. Damit ist bei Kategorie sieben bereits der Vergleichspfad identifiziert. Eine zusätzliche Geräteauswahl per PLIC-Claim entfällt, die Timerursache muss trotzdem behandelt werden. Der Abschnitt über Zeitquelle und Timertermine entwickelt diese Register genauer.
Unten melden UART, Sensor und Taster ihre Anforderungen an den PLIC. UART bedeutet Universal Asynchronous Receiver/Transmitter, eine serielle Sende- und Empfangsschnittstelle. Der Controller bündelt die Anforderungen und benachrichtigt den Hart mit Kategorie elf, Machine External Interrupt. Aus dieser gemeinsamen Kategorie geht noch nicht hervor, welches der drei Geräte bearbeitet werden soll.
Der zurück zum PLIC führende Pfeil ist keine weitere Interruptleitung, sondern ein Softwarezugriff: Der Handler liest über eine Geräteadresse das Claim-Register. Claim bedeutet Übernehmen. Der Zugriff liefert die Identifikationsnummer der ausgewählten Quelle, die Geräte-ID, und besitzt einen Seiteneffekt. Seine genaue Wirkung wird im PLIC-Abschnitt erläutert. Die Pfeile stehen für Informations- und Zugriffswege und nicht für Zeitdauern oder gemessene Latenzen.
Die klassische Organisation unterscheidet nur wenige Maschinen-Interruptkategorien, insbesondere Software, Timer und extern. Das begrenzt nicht die Zahl der Peripheriegeräte und ist auch keine pauschale Aussage über die Zahl physischer Prozessoranschlüsse. Ein Mikrocontroller kann Dutzende externe Quellen haben. Der PLIC sammelt ihre Anforderungen und führt sie für den betrachteten Machine-Mode-

Kontext auf eine gemeinsame externe Benachrichtigung zusammen. Der Kontext ist das Ziel dieser Benachrichtigung; im Lehrmodell betrifft es den Hart im Machine Mode.
Die klassische PLIC-Spezifikation sieht bis zu 1023 Quellen vor. Dies ist eine Obergrenze und keine Zusage über die tatsächliche Ausstattung eines Chips. Eine externe Geräte-ID sieben bliebe eine Quelle der Kategorie elf; sie würde nicht zum lokalen Timer der Kategorie sieben. Die Geräteauswahl liegt beim Controller und beim zugehörigen Claim-Zugriff.
In einem fest verdrahteten System ist die Zuordnung von Gerät und PLIC-ID durch dessen Anbindung im Chip bestimmt und in der Plattformbeschreibung dokumentiert. Die Software wählt diese Nummer nicht frei. In der Grafik sind UART als Quelle eins, Sensor als Quelle zwei und Taster als Quelle drei angenommen. Eine andere Plattform darf sie anders zuordnen. Einstellbar sind dagegen Prioritäten innerhalb der unterstützten Werte; sie bestimmen später die Auswahl zwischen wartenden Quellen.
ID null ist reserviert und bezeichnet kein Gerät. Ein Claim mit Ergebnis null hat für den betrachteten Kontext keine Quelle übernommen. Die zweite Identifikationsstufe liefert somit entweder die ID für die konkrete Bearbeitung oder den Hinweis auf eine fehlende Übernahme; `mcause` enthält weiterhin nur die gemeinsame Kategorie.

Folie 20 / 56

Direct / Vectored: erster Einsprung

2 / 5 · Trap-Eintritt
`mtvec`: BASE in den oberen Bits, MODE in Bits $1{:}0$. MODE 0: Direct, MODE 1: Vectored.

Modus	Eintrittsadresse
Direct	BASE für alle Traps
Vectored, Interrupt	BASE $+ 4c$
Vectored, Exception	BASE

c: Code ohne Interrupt-Bit. Slotabstand: 4 Byte.
BASE `0x80000100`: Timer ($c = 7$) → `0x8000011C`.

- Am Slot steht ein Befehl, etwa `j timer_entry`, kein geladener Zeiger
- Externe Geräte teilen Kategorie 11; PLIC-Claim bleibt nötig

Partnerfrage: Code 11 als Interrupt? Als Exception (`ecall` im Machine Mode)?

Notizen

Neben der Ursache muss der Prozessor beim Trap wissen, wo die Behandlung beginnt. Direct und Vectored bestimmen lediglich die Adresse ihres ersten ausgeführten Befehls. Sicherung, weitere Verzweigungen und Gerätebearbeitung bleiben Aufgaben der Software. Ein Einsprungmodus ist deshalb noch kein vollständiger Handler.
Die Einsprungregel steht in `mtvec`, Machine Trap-Vector Base Address. Dieses CSR, Control and Status Register, kodiert die Basisadresse BASE in den oberen Bits und den Modus MODE in den untersten beiden Bits: null für Direct, eins für Vectored. BASE ist mindestens auf vier Byte ausgerichtet; ihre beiden niedrigsten Adressbits sind daher null. An diesen Positionen können im Register stattdessen Modusbits

stehen. Mit BASE `0x80000100` ergibt sich für Direct der Registerwert `0x80000100`, für Vectored `0x80000101`. Die Hardware trennt Basis und Modus, anstatt zur ungeraden Adresse des zweiten Registerwerts zu springen.
Diese Ausrichtung betrifft die Trap-Basis. Sie bedeutet nicht, dass alle RISC-V-Befehle vier Byte lang oder vier Byte ausgerichtet wären. Mit komprimierten Befehlen sind auch zwei Byte lange Befehle und entsprechende Befehlsgrenzen möglich. Eine konkrete Implementierung kann strengere BASE-Ausrichtung verlangen oder nur bestimmte Modi unterstützen. Die folgende Betrachtung benötigt zunächst die Trennung von Basis und Modus, noch keinen vollständigen Tabellenaufbau.
Im Direct-Modus führt jeder betrachtete Trap zuerst nach BASE. Dort beginnt ein gemeinsamer Dispatcher, eine Routine zur Auswahl der passenden Behandlung. Er liest `mcause`, Machine Cause, und trennt Interrupt-Bit und Ursachencode. Bei externen Interrupts folgt die konkrete Geräteauswahl beim PLIC, Platform-Level Interrupt Controller. Direct verlangt nicht, alle Ursachen oder Gerätestatus nacheinander durchzuprobieren: Nach Prüfung des Interrupt-Bits kann der Code beispielsweise als Index einer geeigneten Software-Sprungtabelle dienen. Diese Tabelle enthält Verzweigungsziele und ist von den ausführbaren Befehlen an Hardware-Vektorplätzen zu unterscheiden. Unbekannte oder nicht unterstützte Codes benötigen eine definierte Behandlung.
Im Vectored-Modus berechnet Hardware bei einem Interrupt den ersten Einsprung als BASE $+ 4c$. Der Code c enthält dabei nicht das höchste Interrupt-Bit. Für den Timer gilt $c = 7$, also $4 \cdot 7 = 28 = \texttt{0x1C}$ Byte. Mit der Beispielbasis liegt der erste Befehl bei `0x8000011C`. Ausgewählte Plätze lassen sich so veranschaulichen:

```
0x80000100: j exception_entry  # BASE: alle Exceptions
0x8000010C: j software_entry   # Interrupt-Code 3
0x8000011C: j timer_entry      # Interrupt-Code 7
0x8000012C: j external_entry   # Interrupt-Code 11
```

Dies ist eine Adressskizze, kein vollständiges Assemblerprogramm. Nicht aufgeführte Zwischenplätze dürfen in einer realen Tabelle nicht einfach entfallen. `j` bezeichnet einen unbedingten Sprung ohne Vorbereitung eines normalen Funktionsaufrufs. Die Zielnamen benennen gedachte Eintrittsroutinen und keine bereits implementierten Repository-Funktionen. Kategorie drei steht für den klassischen Maschinen-Softwareinterrupt, sieben für den Timer und elf für die externe Maschinenkategorie.
Der Abstand zwischen Vektorplätzen beträgt vier Byte. In der Skizze wird jeder Platz durch einen vier Byte langen Sprung genutzt. Allgemein könnten innerhalb dieses Abstands auch zwei komprimierte Zwei-Byte-Befehle stehen. Maßgebend sind die vorgeschriebenen Eintrittsadressen. Assembler und Linker dürfen durch Übersetzung oder Platzierung diese Abstände nicht unbeabsichtigt verändern. Die Skizze ersetzt weder diese Platzierungsarbeit noch die Kontextsicherung.
Exceptions gehen auch im Vectored-Modus stets nach BASE. Das ist eine Regel der betrachteten RISC-V-Architektur, keine Folgerung daraus, dass Exceptions angeblich selten oder grundsätzlich wenig zeitkritisch seien. An BASE kann Software die passende Exception-Behandlung auswählen. Ihre Codes bilden einen anderen Namensraum als die Interrupt-Codes und dürfen dieselben Zahlen verwenden.
Ein Beispiel ist `ecall`, Environment Call, eine bewusst ausgelöste Anforderung an die Ausführungsumgebung. Im Machine Mode erzeugt dieser Befehl eine Exception mit Code elf. Der externe Maschineninterrupt besitzt ebenfalls Code elf, aber sein Interrupt-Bit ist eins. Die `ecall`-Exception springt daher nach BASE, also `0x80000100`; der externe Interrupt springt nach BASE $+ 4 \cdot 11$, also `0x8000012C`. Wer beide an dieselbe Stelle legt, übersieht entweder das Interrupt-Bit oder die Exception-Regel. Die überlappenden Codes erklären die nötige Unterscheidung, beweisen aber nicht, dass jede denkbare Architektur dieselbe Vektorisierung wählen müsste. Rein rechnerisch läge auch Interrupt-Code null bei BASE; im hier verwendeten klassischen Maschinenmodell ist dies keine regulär belegte Interrupt-Kategorie.
Aus der bekannten Cortex-M-Organisation darf die Bedeutung einer Vektortabelle nicht ungeprüft über-

tragen werden. Deren Handler-Einträge enthalten Adressen, welche die Hardware lädt. Bei der hier betrachteten RISC-V-Vektorisierung stehen an den Eintrittsplätzen dagegen Befehle, die ausgeführt werden, typischerweise Sprünge zur eigentlichen Behandlung. Ein Funktionszeiger wird dort nicht automatisch geladen.
Vectored trennt außerdem Kategorien und keine einzelnen Geräte hinter dem PLIC. Alle externen Quellen teilen den Platz für Code elf. Die dortige Routine benötigt weiterhin Claim, den Übernahmezugriff auf den PLIC, um eine konkrete Geräte-ID zu bekommen; auch die Gerätequittierung bleibt nötig. Selbst beim Cortex-M-NVIC, Nested Vectored Interrupt Controller, können mehrere Geräte- oder Unterursachen einen Vektor teilen. Als weiterführender Begriff bezeichnet CLIC den Core Local Interrupt Controller, eine alternative RISC-V-Interruptorganisation unter anderem mit quellenbezogener Vektorisierung. Dessen weitere Verfahren und konkreter Versionsstand werden hier nicht vorausgesetzt und ändern die erklärten Regeln für `mtvec` und klassischen PLIC nicht.
Bei passenden Eintrittsroutinen kann Vectored die erste Kategorienauswahl vorwegnehmen und Auswahlbefehle eines gemeinsamen Dispatchers sparen. `mcause` kann trotzdem für weitere Unterscheidungen oder Diagnose benötigt werden. Der tatsächliche Zeitgewinn hängt vom Code und von der Plattform ab; der Modus belegt allein keine feste Reaktionszeit. Insbesondere beseitigt er weder gesperrte Interrupts noch eine bereits laufende, nicht verschachtelte ISR, Interrupt Service Routine. Die später im Reaktionsbudget angenommenen $40\ \mu$s Blockierung bleiben deshalb ein eigener Zeitanteil. Aus diesem Lehrwert folgt keine allgemeine Größenordnung für das Verhältnis zwischen Blockierung und eingesparten Befehlszeiten.

Folie 21 / 56

Den Trap-Pfad in eigenen Worten erklären

2 / 5 · Trap-Eintritt

RV32: `mcause = 0x8000000B`, später liefert Claim die ID 2.

1. Welche Information liefert jeweils der erste und der zweite Wert?
2. Was hat die Hardware beim Eintritt bereits gesichert?
3. Was muss Software vor der Fortsetzung erledigen?

Verwenden Sie die Begriffe Ursache, Kontext und Gerätebearbeitung.

Notizen

Die bisher getrennt erklärten Schritte lassen sich nun zu einem vollständigen Trap-Pfad verbinden. Im RV32-Beispiel enthält `mcause`, Machine Cause, den Rohwert `0x8000000B`. Das höchste Bit ist eins, der Code elf: Es liegt ein Interrupt der externen Maschinenkategorie vor. Die später aus Claim gelieferte Zahl zwei bezeichnet dagegen die konkrete Quelle am PLIC, Platform-Level Interrupt Controller. Sie ist keine andere Schreibweise desselben Ursachenwerts.
Zum Zeitpunkt des Softwareeintritts hat Hardware unter anderem die Fortsetzungsadresse in `mepc`, Machine Exception Program Counter, und die Ursache in `mcause` erhalten. Sie hat die globale Maschinen-Interruptfreigabe gesperrt und deren vorherigen Zustand festgehalten. Allgemeine Register mit noch benötigten Werten des Hauptprogramms liegen dadurch nicht automatisch auf dem Stack. Ihr Erhalt gehört weiterhin zur Aufgabe der Software und zum für die Fortsetzung nötigen Kontext.
Software sichert daher ihre benötigten Register, wählt den zur Ursache passenden Gerätepfad und bearbeitet die übernommene Quelle vollständig. Zum externen Pfad gehört anschließend auch der Control-

lerabschluss. Danach werden die Software-Sicherungen wiederhergestellt. `mret`, Machine Return, kehrt anhand des erhaltenen Trap-Zustands zurück. Jede Teilaufgabe hat einen eigenen Zweck: Ursachenwahl verhindert die falsche Behandlung, Geräteservice erledigt die anstehende Arbeit, und Kontexterhalt schützt die unterbrochene Ausführung.
Damit ist der Prozessorpfad beschrieben, die eigentliche externe Gerätebearbeitung aber noch nicht im Detail geöffnet. Im nächsten Abschnitt wird untersucht, wie der PLIC eine Quelle auswählt, was Claim verändert und weshalb Geräteursache und Controllerabschluss getrennte Handlungen verlangen.

Folie 22 / 56

3 · Externe Geräte behandeln

Welches Gerät wartet – und wann ist seine Anforderung abgeschlossen?
Ereignisse → Trap-Eintritt → Geräte → Timer → Reaktionszeit
PLIC: Quelle auswählen, Geräteursache behandeln, Übernahme abschließen.

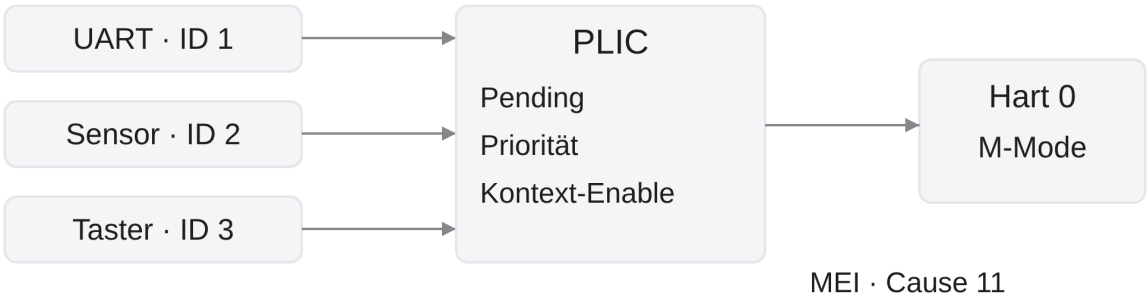
Notizen

Der Trap-Eintritt liefert bei einem externen Interrupt zunächst eine Kategorie. Welche UART, welcher Sensor oder welcher Taster die Behandlung benötigt, ist damit noch nicht entschieden. Der Platform-Level Interrupt Controller, kurz PLIC, sammelt diese externen Anforderungen und macht sie einem Empfänger zugänglich. Ein solcher Kontext ist durch Hart und Privilegstufe bestimmt; das Lehrmodell verwendet weiterhin genau einen Hart im Machine Mode.
Die neue Frage betrifft nicht nur die Rangfolge der Geräte. Eine vollständige Behandlung muss drei verschiedene Zustände verändern: die wartende Controlleranforderung, die eigentliche Geräteursache und die Sperre gegen eine weitere Weiterleitung derselben Quelle. Deshalb bilden Claim, Geräteaktion und Complete eine zusammenhängende, aber nicht austauschbare Folge.
Dieser Abschnitt öffnet den bisher als Verteiler betrachteten Controller und verfolgt den gesamten Lebenszyklus einer Anforderung. Priorität erklärt, welche wartende Quelle als Nächstes gewählt wird. Sie erklärt im nicht verschachtelten Grundmodell noch keine Unterbrechung einer bereits laufenden Behandlung. Damit entsteht zugleich die Verbindung zur späteren Reaktionszeit: Auswahlregeln und unvermeidbare Wartephasen müssen gemeinsam betrachtet werden.

Folie 23 / 56

Der PLIC verbindet Geräte mit einem Kontext

3 / 5 · Geräte und PLIC



- Quellen haben Geräte-IDs und Prioritäten
- Ein Kontext gehört zu Hart und Privilegstufe
- Globale Ursache im Lehrmodell: Machine External, Code 11

Notizen

Nachdem der Trap-Eintritt und die Unterscheidung seiner Ursachen geklärt sind, lässt sich der externe Gerätepfad genauer verfolgen. Der PLIC, Platform-Level Interrupt Controller oder Plattform-Interrupt-Controller, sammelt Anforderungen mehrerer Geräte und stellt sie geeigneten Empfängern zu. Er verwaltet dafür zwei unterschiedliche Ebenen: Zustand und Einstellungen einer Gerätequelle sowie Einstellungen ihres Empfängers, des Kontexts.

In der Abbildung führen die Wege von links nach rechts von UART, Sensor und Taster über den PLIC zu Hart null im Machine Mode. UART steht für Universal Asynchronous Receiver/Transmitter, eine serielle Sende- und Empfangsschnittstelle. Die Geräte tragen im Beispiel die IDs eins, zwei und drei. Diese Identifikationsnummern ergeben sich aus ihrer Anbindung an die Plattform. Die Pfeile bezeichnen Informationswege; sie legen weder Bearbeitungszeiten noch Geräteadressen fest.

Zu jeder Quelle gehören eine ID, eine von der Software einstellbare Priorität innerhalb der unterstützten Werte und ein Pending-Bit. Dieses Bit zeigt an, dass eine Controlleranforderung wartet. Es gehört zur Quelle und wird gemeinsam verwaltet, nicht für jeden möglichen Empfänger getrennt gespeichert. An der Quellanbindung sitzt außerdem ein Gateway. Es übersetzt das Gerätesignal in eine Anforderung und lässt pro Quelle höchstens eine noch nicht abgeschlossene Anforderung zum PLIC weiter. Anschließend wartet es auf deren Abschluss. Ob zwischenzeitlich weitere Ereignisse gespeichert werden können, bestimmen Gerät und Gateway; aus der Controllerfunktion folgt keine unbegrenzte Ereigniswarteschlange.

Ein Kontext ist dagegen ein Empfänger, der einem Hart und einer Privilegstufe zugeordnet ist. Ein Hart ist ein Hardware-Ausführungskontext mit eigenem Befehlsstrom. Das Lehrmodell verwendet genau einen PLIC-Kontext: Hart null im Machine Mode, der höchsten RISC-V-Privilegstufe. In der Grafik sind dafür die Kurzformen M-Mode und MEI angegeben. MEI bedeutet Machine External Interrupt, die externe Maschinen-Interruptkategorie mit Code elf.

Jeder Kontext besitzt eigene Enable-Bits, also Freigaben für die einzelnen Quellen, einen Threshold genannten Schwellenwert und einen Claim/Complete-Zugriff. Der Schwellenwert bleibt im Hauptteil null. Ein Lesen des Claim/Complete-Registers übernimmt eine Gerätequelle; ein Schreiben der übernommenen ID meldet deren Abschluss. Die beiden Aktionen werden als Claim und Complete bezeichnet.

Eine Matrix macht die Aufteilung anschaulich: Die Quellen bilden die Zeilen, die Kontexte die Spalten.

An jedem Kreuzungspunkt steht ein Enable-Bit, das diese Zuordnung freigibt oder sperrt. Priorität und Pending-Zustand gehören zur ganzen Quellenzeile, der Schwellenwert zur Empfängerspalte. Ein gesetztes Enable-Bit reicht deshalb für eine Benachrichtigung nicht aus. Zusätzlich müssen Pending-Zustand, Priorität und Schwellenwert passen; auf der Hart-Seite gelten weitere Freigaben.
Die Bezeichnung Kontext ist präziser als Kern, weil ein Hart externe Interrupts auf verschiedenen Privilegstufen empfangen kann. Eine angenommene Plattform mit zwei Harts und je einem Machine- und Supervisor-Kontext besitzt beispielsweise $2 \cdot 2 = 4$ Kontexte. Der Supervisor Mode ist eine unterhalb des Machine Mode liegende Privilegstufe für Betriebssystemsoftware. Bei passender Plattformkonfiguration und Weiterleitung könnte eine UART das Betriebssystem auf Hart eins benachrichtigen, ein Sensor dagegen Firmware auf Hart null. Machine External Interrupt verwendet Code elf, Supervisor External Interrupt Code neun. Vier Kontexte sind dabei eine Eigenschaft der angenommenen Plattform, keine allgemeine Zusage für jedes Zwei-Hart-System. Im Folgenden bleibt es beim einzelnen Machine-Mode-Kontext; an ihm lässt sich nun die Auswahl zwischen mehreren wartenden Quellen untersuchen.

Folie 24 / 56

Welche externe Quelle kommt zuerst?

3 / 5 · Geräte und PLIC
Alle Quellen warten und sind freigegeben. Der Schwellenwert ist 0.

Geräte-ID	Quelle	Priorität
1	UART	3
2	Sensor	7
3	Taster	7

- Höhere Priorität zuerst, bei Gleichstand kleinere ID
- Priorität 0 wäre unwirksam
- Bestimmen Sie die Reihenfolge ohne neue Ereignisse

Notizen

Die Quellenpriorität beantwortet eine Auswahlfrage: Welche der bereits wartenden externen Anforderungen soll der PLIC als Nächstes an den Empfänger vergeben? Geräte-ID und Priorität erfüllen dabei unterschiedliche Aufgaben. Die ID identifiziert die Quelle, die Priorität ordnet ihre Dringlichkeit. Eine höhere Zahl bedeutet im betrachteten Controller eine höhere Priorität; Priorität null trägt nicht zur Benachrichtigung bei.
Im Beispiel warten alle drei Quellen und sind für den einzelnen Kontext freigegeben. Bei seinem Schwellenwert null liegen die positiven Prioritäten drei und sieben oberhalb der Schwelle. UART, Universal Asynchronous Receiver/Transmitter, hat ID eins und Priorität drei. Sensor und Taster besitzen die IDs zwei und drei bei gleicher Priorität sieben. Zuerst gewinnt deshalb der Sensor: Seine Priorität ist maximal, und bei Gleichstand entscheidet die kleinere Geräte-ID. Werden die Quellen jeweils vollständig bearbeitet und abgeschlossen und treten keine neuen Ereignisse ein, folgen Taster und UART. Die Reihenfolge lautet zwei, drei, eins.
Diese Rangfolge gilt für die Auswahl wartender Quellen. Sie verkürzt keine bereits laufende, nicht ver-

schachtelte Behandlung und erlaubt ihr auch nicht automatisch, eine andere Behandlung zu unterbrechen. Die Reaktionszeit hängt somit nicht nur von der Prioritätszahl ab. Wie die ausgewählte Quelle tatsächlich übernommen wird, erklärt der Claim-Zugriff. Ein veränderter Schwellenwert und ein bewusst ohne Benachrichtigung ausgeführter Claim bleiben der optionalen Vertiefung vorbehalten.

Folie 25 / 56

Claim ist ein Zugriff mit Seiteneffekt

3 / 5 · Geräte und PLIC

- Liefert die höchstpriorisierte claimbare Geräte-ID
- Löscht atomar das zugehörige PLIC-Pending
- ID 0: keine claimbare Quelle; kein Handler und kein Complete
- Die Ursache im Gerät ist dadurch noch nicht bearbeitet

Notizen

Die bisher betrachtete Auswahl wird durch einen Claim wirksam. Claim bedeutet hier die Übernahme einer wartenden Quelle beim PLIC, Platform-Level Interrupt Controller. Dazu liest die Software das zu ihrem Kontext gehörende Claim-Register. Dieser Lesezugriff fragt nicht nur einen Zustand ab, sondern verändert ihn: Der PLIC liefert die ID der höchstpriorisierten wartenden, für den Kontext freigegebenen Quelle mit positiver Priorität und löscht zugleich deren Pending-Bit. Auswahl und Entfernen der Wartemarkierung bilden einen atomaren, also unteilbaren Schritt.
Das Claim-Register ist ein MMIO-Register. MMIO steht für Memory-Mapped Input/Output, den Gerätezugriff über Speicheradressen. Ein solcher Zugriff muss sich nicht wie das Lesen gewöhnlichen Speichers verhalten. Eine UART, Universal Asynchronous Receiver/Transmitter, kann beispielsweise ein Empfangsregister besitzen, dessen Lesen ein Byte aus dem Empfangspuffer entnimmt. Ein weiterer Zugriff liefert dann möglicherweise das nächste Byte. Welche Seiteneffekte ein konkretes UART-Register hat und was bei leerem Puffer geschieht, legt die Gerätebeschreibung fest.
Entsprechend führt auch jedes erneute Lesen des Claim-Registers einen neuen Claim aus. Solange die gerade übernommene Quelle noch nicht abgeschlossen ist, erhält die Software eine andere wartende Quelle oder null, nicht erneut eine unveränderte Kopie des ersten Ergebnisses. Die einmal gelesene ID muss deshalb gespeichert und für die gesamte Gerätebearbeitung einschließlich Abschluss verwendet werden.
Die Atomizität wird besonders bei mehreren Harts sichtbar. Sind die Benachrichtigungsbedingungen zweier Kontexte erfüllt und ist dieselbe Quelle für beide freigegeben, können beide Empfänger benachrichtigt werden. Lesen anschließend beide Harts ihren Claim, übernimmt nur einer die gemeinsame wartende Anforderung. Der andere erhält null oder eine andere Quelle. Damit wird dieselbe PLIC-Anforderung nicht gleichzeitig doppelt vergeben. Daraus folgt jedoch weder, dass zuvor beliebig viele physische Einzelereignisse erfasst wurden, noch, dass die Geräteursache bereits bearbeitet ist. Die Grenzen eines einzelnen Bereitschaftsbits gelten weiterhin.
ID null bezeichnet kein Gerät. Ein Handler kann nach einer Benachrichtigung eintreten und trotzdem null erhalten, etwa weil ein anderer Hart die Quelle inzwischen übernommen hat. Zwischen Benachrichtigung und Claim kann sich die Lage verändern. Für null entfallen Gerätebearbeitung und Complete; die Software stellt lediglich den nötigen Kontext wieder her und folgt ihrem korrekten Trap-Rückkehrpfad.
Claim verändert nur die Wartemarkierung im Controller. Empfangsdaten können weiterhin im UART-Puffer

liegen, und ein Gerätestatus kann weiterhin aktiv sein. Deshalb sind drei Stellen mit jeweils eigener Verantwortung zu unterscheiden:

Stelle	Was dort ansteht	Passender Schritt
PLIC	Pending-Bit der Quelle	Claim
Gerät	Statusflag und Daten	Gerätebehandlung nach Datenblatt
Gateway	offene Weitergabe; weitere Anforderung gesperrt	Complete derselben übernommenen ID

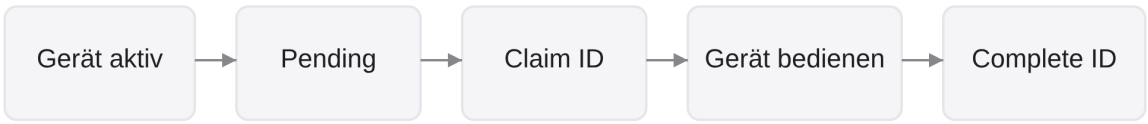
Das Gateway wurde bereits durch die Weitergabe belegt. Claim allein öffnet es nicht. Erst Complete erlaubt die nächste Weitergabe; ist die Geräteursache weiterhin aktiv, kann sie danach erneut gemeldet werden. Das Entfernen des PLIC-Pending und die Quittierung im Gerät sind somit keine austauschbaren Aktionen. Aus dem Seiteneffekt ergibt sich eine wichtige Debugger-Falle. Ein automatisch aktualisiertes Beobachtungsfenster kann durch tatsächliches Lesen des Claim-Registers eine Quelle übernehmen. Wird deren ID anschließend nicht für die normale Bearbeitung und Complete verwendet, wartet das Gateway weiter auf den Abschluss. Das Pending-Bit ist zwar verschwunden, weitere Anforderungen dieser Quelle werden aber nicht zum PLIC weitergereicht. Das Gerät kann weiterarbeiten, während seine Interruptquelle scheinbar verstummt. Der Zustand bleibt bestehen, bis der fehlende passende Abschluss nachgeholt oder der betroffene Controllerzustand zurückgesetzt wird.
Auch eine Diagnoseausgabe kann dieses Problem erzeugen: Liest ein Argument für die C-Funktion `printf` erneut das Claim-Register, kann eine weitere Quelle ohne Abschluss übernommen werden. Die bereits gespeicherte ID auszugeben, verursacht dagegen keinen neuen Claim. Die praktische Regel lautet daher, das Register genau einmal für die beabsichtigte Übernahme zu lesen, die ID aufzubewahren und genau diese ID später abzuschließen. Damit sind alle Schritte für einen vollständigen Gerätepfad vorbereitet.

Folie 26 / 56

Claim, Gerät bearbeiten, Complete

3 / 5 · Geräte und PLIC

Normaler Lebenszyklus einer bekannten Gerätequelle



PLIC-Pending gelöscht

Geräteursache bearbeiten, danach dieselbe ID abschließen.

Claim und Complete betreffen den Controller; Gerätequittierung ist ein eigener Schritt.

Notizen

Eine vollständige externe Behandlung verbindet die bisher getrennten Verantwortlichkeiten zu einem Lebenszyklus. Zuerst meldet ein Gerät seine Ursache an das Gateway, seine Quellanbindung. Der

PLIC, Platform-Level Interrupt Controller, speichert die Anforderung als Pending und benachrichtigt bei passenden Freigaben den Hart. Die Software übernimmt die Geräte-ID durch genau einen Claim; dadurch wird das zugehörige PLIC-Pending gelöscht.
Anschließend folgt die eigentliche Gerätearbeit. Bei einer UART, Universal Asynchronous Receiver/Transmitter, kann dazu das Lesen empfangener Daten gehören. Andere Geräte benötigen andere Zugriffe, um Daten zu übernehmen und die Ursache zurückzunehmen. Maßgeblich ist jeweils die Gerätebeschreibung; ein universelles Registerrezept gibt es dafür nicht. Erst danach schreibt die Software dieselbe übernommene ID als Complete zurück.
Mit dem Abschluss kann das Gateway wieder Anforderungen dieser Quelle weiterreichen. Die Pfeile in der Ablaufdarstellung geben diese Reihenfolge und die Zuständigkeiten wieder, keine garantierten Taktdauern. Weder Claim noch Complete ersetzt die Gerätebearbeitung. Ebenso informiert ein Funktionsrücksprung den PLIC nicht automatisch über den Abschluss. Besonders deutlich wird die Trennung bei einem Gerät, dessen Anforderungsleitung so lange aktiv bleibt, wie ihre Ursache besteht.

Folie 27 / 56

Level-Ursache bleibt aktiv

3 / 5 · Geräte und PLIC

1. Ein Gerät hält seine Anforderungsleitung aktiv
2. Claim entfernt das PLIC-Pending
3. Software meldet Complete, ohne die Geräteursache zu beseitigen
4. Das Gateway kann erneut Pending melden

Priorität erhöhen behebt die Geräteursache nicht.

Notizen

Eine Level-Anforderung wird durch einen anhaltenden Leitungszustand angezeigt. Sie bleibt aktiv, solange die zugehörige Geräteursache besteht. Eine Edge-Anforderung beruht dagegen auf einer Flanke; wie solche Flanken erfasst und gespeichert werden, hängt von Gerät und Gateway ab. Das hier betrachtete Beispiel verwendet bewusst eine Level-Anforderung.
Nach dem Claim ist das Pending-Bit im PLIC, Platform-Level Interrupt Controller, zunächst gelöscht. Unterlässt die Software jedoch die passende Gerätebearbeitung oder Quittierung, bleibt die Anforderungsleitung aktiv. Complete schließt lediglich die übernommene Controlleranforderung ab und macht das Gateway wieder aufnahmebereit. Es kann den weiterhin aktiven Leitungszustand deshalb erneut als Anforderung an den PLIC weitergeben. Wiederholte Handler-Eintritte sind in diesem Ablauf die Folge einer nicht beseitigten Ursache, obwohl die Claim-Zugriffe selbst korrekt ausgeführt wurden.
Weder eine höhere Priorität noch ein anderer Vektorplatz noch allein `mret`, die Trap-Rückkehr, löst dieses Problem. Abhilfe schafft die nach Gerätebeschreibung erforderliche Ursachenbehandlung. Ebenso wenig darf ein Pending-Bit als unbegrenzter Speicher für einzelne Flanken verstanden werden. Die Controllerwartemarkierung und ein Datenpuffer im Gerät erfüllen verschiedene Aufgaben. Eine korrekte externe Behandlung muss daher kurz genug für ihre zeitlichen Anforderungen sein und trotzdem alle notwendigen Schritte vollständig ausführen.

Folie 28 / 56

Normaler externer Behandlungspfad

3 / 5 · Geräte und PLIC

Pseudocode für bekannte, konfigurierte Quellen:

```
id = einmal Claim lesen
wenn id = 0: keine Gerätebearbeitung
sonst:
    Daten / Status des Geräts id bearbeiten
    Geräteursache nach Gerätebeschreibung quittieren
    dieselbe id als Complete schreiben
Kontext wiederherstellen und Trap-Rückkehr
```

Notizen

Der Pseudocode fasst den Lebenszyklus einer bekannten und konfigurierten Gerätequelle zusammen. Die Variable `id` speichert deren Identifikationsnummer. Der Claim-Zugriff beim PLIC, Platform-Level Interrupt Controller, wird genau einmal ausgeführt. Eine zweite diagnostische Lesung wäre eine weitere Übernahme und könnte eine zusätzliche Quelle liefern. Für das Ergebnis null folgen weder Gerätebearbeitung noch Complete, weil keine Quelle übernommen wurde.
Bei einer gültigen bekannten ID übernimmt die Software die erforderlichen Daten und behandelt den Gerätestatus. Quittieren bedeutet, die bearbeitete Ursache auf die für dieses Gerät vorgesehene Weise zurückzunehmen. Das kann je nach Gerät ein bestimmter Lese- oder Schreibzugriff sein. Anschließend meldet Complete genau die zuvor übernommene ID beim Controller als abgeschlossen. Zum Schluss wird der gesicherte Kontext wiederhergestellt; `mret`, Machine Return, stellt den Trap-Rückkehrzustand her.
Jeder dieser Schritte verhindert eine andere Fehlerklasse. Ohne Datenübernahme können Nutzdaten fehlen. Ohne passende Quittierung kann die Geräteursache aktiv bleiben und erneut Interrupts erzeugen. Ohne Complete bleibt die Quellanbindung für die nächste Anforderung belegt. Ohne Kontextwiederherstellung kann das unterbrochene Hauptprogramm mit beschädigten Registerwerten fortsetzen. Die Reihenfolge ist deshalb eine begründete Behandlungskette und keine beliebig verkürzbare Aufzählung.
Vorausgesetzt sind ein korrekt vorbereiteter Ein- und Austritt sowie bekannte, freigegebene Quellen. Unerwartete IDs benötigen in realer Software eine definierte Fehlerbehandlung. Der Pseudocode erklärt den Ablauf und ist kein vollständiger Treiber. Offen bleibt außerdem, wie sich andere währenddessen eintreffende Anforderungen auf die Reaktionszeit auswirken.

Folie 29 / 56

Hohe Priorität unterbricht nicht automatisch

3 / 5 · Geräte und PLIC

Während einer ISR ist die globale Freigabe MIE im Grundmodell 0.

- Eine weitere Anforderung kann trotzdem entstehen und warten
- PLIC-Priorität ordnet wartende externe Quellen
- Sie unterbricht die laufende ISR nicht von selbst
- Lange ISR verlängern deshalb mögliche Wartezeiten

Unser Hauptmodell bleibt kurz und nicht verschachtelt.

Notizen

Eine Quellenpriorität legt fest, welche wartende externe Anforderung als Nächstes ausgewählt wird. Sie bewirkt noch keine Unterbrechung einer bereits laufenden Behandlung. Im vereinbarten Ein-Hart-Modell sind Interruptbehandlungen nicht verschachtelt: Beim Trap-Eintritt setzt die Hardware MIE auf null, und die Software lässt es während der ISR dabei. ISR steht für Interrupt Service Routine, Unterbrechungsbehandlung. MIE, Machine Interrupt Enable, ist die globale Maschinen-Interruptfreigabe in `mstatus`.
Neue maskierbare Maschinen-Interruptanforderungen können in dieser Zeit entstehen und gegebenenfalls gespeichert bleiben. Ein weiterer Interrupt-Trap wird im Grundmodell aber nicht angenommen. Eine höher priorisierte Sensoranforderung kann deshalb die Restdauer einer bereits laufenden UART-Behandlung abwarten müssen. Die Prioritäten im PLIC, Platform-Level Interrupt Controller, entscheiden erst wieder bei der nächsten Auswahl zwischen wartenden externen Quellen. Auch die direkte lokale Anbindung eines Timers verschafft ihm nicht automatisch Vorrang vor der laufenden ISR.
Hier unterscheidet sich das Modell von einer möglichen STM32-Konfiguration aus dem Labor. Der NVIC, Nested Vectored Interrupt Controller, im Cortex-M kann bei geeigneten Freigaben und Prioritäten eine laufende Behandlung durch höher priorisierte Arbeit unterbrechen. Eine höhere PLIC-Quellenpriorität allein stellt diesen Mechanismus nicht bereit.
Software könnte Interrupts innerhalb einer Behandlung wieder freigeben. Dafür müssten jedoch zusätzlicher Rückkehrzustand, Auswahlregeln und Stackverbrauch beherrscht werden. Diese eigene Entwurfsentscheidung bleibt der optionalen Vertiefung vorbehalten. Im Hauptmodell ist die erste wirksame Maßnahme, die unmittelbar nötige Gerätearbeit kurz zu halten und längere Auswertung aufzuschieben. Nach dem externen Gerätepfad lässt sich nun dieselbe Frage für zeitgesteuerte Arbeit stellen: Woher stammt eine Anforderung, wenn nicht ein Gerät, sondern ein geplanter Zeitpunkt ihre Ursache ist?

Folie 30 / 56

4 · Zeit und Termine planen

Wie bleibt ein periodischer Ablauf trotz verspäteter Behandlung im Raster?
Ereignisse → Trap-Eintritt → Geräte → Timer → Reaktionszeit
mtime misst die Zeit; mtimecmp benennt den nächsten Vergleichstermin.

Notizen

Der externe Gerätepfad erklärt, wie ein eingetretenes Ereignis behandelt wird. Der lokale Timer ergänzt einen anderen Fall: Ein Ereignis soll zu einem programmierten Zeitpunkt entstehen. Die Grundideen von Pending, Freigabe und korrekter Rückkehr bleiben erhalten; als Ursache dient nun der Vergleich einer fortlaufenden Hardwarezeit mit einem eingestellten Termin.
Aus dem STM32-Labor ist bereits ein Timer bekannt, dessen ARR die Hardwareperiode festlegt. Der hier betrachtete RISC-V-Systemtimer setzt seine Zeitbasis für eine Anwendungsperiode dagegen nicht zurück. Die Software muss entscheiden, welchen nächsten Vergleichstermin sie einträgt. Damit entstehen zwei getrennte Fragen: Welche Zeit ist tatsächlich vergangen, und auf welchem Raster sollen zukünftige Aktionen liegen?
Der vierte Abschnitt entwickelt diese Unterscheidung von Zeitquelle, Termin und Handler-Aufruf. Er zeigt, wie relative Planung Verspätungen übernehmen kann, wie ein absolutes Sollraster erhalten bleibt und was bei mehreren fälligen Terminen noch nachholbar ist. Ein nachgerechneter Termin liefert allerdings keine verlorenen historischen Messwerte. Zeitplanung und Datenerfassung bleiben auch bei einem korrekt arbeitenden Timer unterschiedliche Aufgaben.

Folie 31 / 56

`mtime` ist die Zeitquelle

4 / 5 · Timer und Sollraster

- Machine Time: 64-Bit-Zähler, auch auf RV32
- Frequenz der Zeitquelle von der Plattform bestimmen
- Kein notwendiger Gleichlauf mit dem Prozessortakt
- Zeit seit Start: Differenz zum gespeicherten Startwert

Notizen

Zeitgesteuerte Arbeit benötigt zunächst eine Zeitbasis, die unabhängig davon weiterläuft, ob das Programm sie gerade auswertet. `mtime`, Machine Time, ist im betrachteten RISC-V-Modell ein vierundsechzig Bit breiter Zähler der Plattformzeit. RV32 bezeichnet dagegen die Variante mit zweiunddreißig Bit breiten allgemeinen Integerregistern. Die Breite dieser Prozessorregister verkürzt `mtime` nicht. Seine konstante Zählfrequenz wird von der Plattform angegeben und muss nicht mit der Prozessorfrequenz übereinstimmen. Der Wert ist nicht automatisch die Zeit seit dem Start eines bestimmten C-Programms. Dafür speichert das Programm einen Startwert t_0 und bildet später die Differenz zur aktuellen Hardwarezeit. Ein Timer-Tick bezeichnet dabei einen Zählschritt der Zeitquelle. Ein periodischer Softwaretick, also ein wiederkehrender

Softwaretermin, ist davon zu unterscheiden.
Die Bedeutung der großen Zählerbreite lässt sich mit der angenommenen Lehrfrequenz von zehn Megahertz, also 10^7 Schritten pro Sekunde, abschätzen. Ein zweiunddreißig Bit breiter Zähler durchläuft seine Werte bereits nach

$$T_{32} = \frac{2^{32}}{10^7\ \mathrm{s}^{-1}} = 429{,}4967296\ \mathrm{s} \approx 7{,}16\ \mathrm{min}.$$

Für vierundsechzig Bit ergibt sich dagegen

$$T_{64} = \frac{2^{64}}{10^7\ \mathrm{s}^{-1}} \approx 1{,}845 \cdot 10^{12}\ \mathrm{s} \approx 58454\ \mathrm{Jahre}.$$

Die Jahresumrechnung verwendet $365{,}25$ Tage pro Jahr und 86400 Sekunden pro Tag. Bei einem Startwert null liegt dieser Umlauf weit außerhalb der kurzen betrachteten Abläufe. Die Terminrechnung darf ihn unter diesen Annahmen ausklammern. Ein bereits nahe der oberen Grenze stehender Zähler wäre anders zu behandeln. Zusätzlich dürfen die arithmetischen Zwischenergebnisse der verwendeten Rechnung nicht überlaufen; die große Registerbreite allein ersetzt diese Voraussetzung nicht.
Beim Zugriff entsteht eine andere Schwierigkeit. `mtime` wird hier über MMIO, Memory-Mapped Input/Output, also Geräteadressen, erreicht. Die einzelnen Zugriffe sind im betrachteten RV32-MMIO-Modell zweiunddreißig Bit breit. Der vollständige Zeitwert muss deshalb aus zwei Hälften zusammengesetzt werden. RV32 allein garantiert keinen atomaren Vierundsechzig-Bit-Gerätezugriff; entscheidend sind die Zugriffsmöglichkeiten der konkreten Plattform.
Läuft die untere Hälfte zwischen den beiden Lesezugriffen über, können eine alte obere und eine neue untere Hälfte einen falschen Mischwert bilden. Eine Abweichung um 2^{32} Zählschritte entspricht bei zehn Megahertz rund $429{,}5$ Sekunden, wiederum gut sieben Minuten. Eine große Gesamtbreite verhindert diesen Fehler also nicht. Die optionale Vertiefung zum konsistenten Lesen von `mtime` kapselt ihn mit der Funktion `read_mtime32`. Im Hauptteil wird eine Plattformfunktion mit dem Vertrag vorausgesetzt, einen konsistenten Zeitwert zu liefern. Auf dieser Grundlage können Zeitunterschiede nun in die richtige Einheit umgerechnet werden.

Folie 32 / 56

Timer-Ticks sind keine Prozessorzyklen

4 / 5 · Timer und Sollraster
Lehrannahmen: Timer $f_T = 10$ MHz, Prozessor $f_C = 100$ MHz.

$$\Delta = f_T \cdot 1\ \mathrm{ms} = 10000\ \text{Timer-Ticks}$$

Größe für 1 ms	Anzahl
Timer-Ticks	10000
Prozessorzyklen	100000

Eine Änderung des Prozessortakts ändert die vorgegebene Timerfrequenz nicht.

Notizen

Ein Zählerwert wird erst durch seine Frequenz zur Zeitangabe. Im Lehrbeispiel läuft die Hardware-Zeitquelle mit $f_T = 10$ Megahertz, der Prozessor dagegen mit $f_C = 100$ Megahertz. Megahertz bedeutet eine Million Schritte beziehungsweise Taktperioden pro Sekunde. Beide Frequenzen beschreiben unterschiedliche Vorgänge und dürfen bei der Umrechnung nicht vertauscht werden.
Ein Timer-Tick dauert hier $1/f_T = 0{,}1\ \mu$s. Eine Millisekunde entspricht deshalb

$$\Delta = f_T \cdot 10^{-3}\ \text{s} = 10^7\ \text{s}^{-1} \cdot 10^{-3}\ \text{s} = 10000\ \text{Timer-Ticks}.$$

Δ ist die Länge des gewünschten Intervalls in Zählschritten. Der Prozessor durchläuft in derselben Millisekunde hunderttausend Taktperioden. Diese Zahl ist keine Zusage über hunderttausend fertig ausgeführte Befehle: Speicherzugriffe und andere Ausführungseigenschaften können mehrere Zyklen beanspruchen. Die Gegenüberstellung trennt ausschließlich die beiden Zeiteinheiten.
Wird der Prozessortakt auf fünfzig Megahertz reduziert, bleiben unter der angenommenen unabhängigen Zeitquelle zehntausend Timer-Ticks pro Millisekunde richtig. Eine anhand von Prozessorzyklen geschätzte Warteschleife hätte dagegen andere Voraussetzungen. Die Rückrechnung lautet $t = n/f_T$, wobei n die Tickzahl und t die Zeit in Sekunden bezeichnet. Eine Terminplanung beginnt somit mit der Frequenz ihrer eigenen Zeitquelle, nicht mit einer zufällig bekannten Prozessorfrequenz. Mit dieser Umrechnung lässt sich als Nächstes ein gewünschter Zeitpunkt in einen Hardwarevergleich übersetzen.

Folie 33 / 56

Vergleich und Freigaben des Timers

4 / 5 · Timer und Sollraster

Im Machine-Mode-Modell müssen alle Bedingungen gelten.



- STM32-Update: ARR erzeugt das Periodenraster in Hardware
- Hier: `mtime` zählt weiter, `mtimecmp` setzt einen einzelnen Termin

Notizen

`mtime` beantwortet die Frage nach der aktuellen Zeit. `mtimecmp`, Machine Time Compare, ergänzt sie um einen Vergleichstermin. Beide Werte sind im betrachteten System vierundsechzig Bit breit und werden vorzeichenlos verglichen. Sobald `mtime` mindestens `mtimecmp` erreicht, steht eine Timeranforderung an. Ein Vergleichswert hundert ist daher bei aktuellem Zählerstand hundert bereits fällig; um die Bedingung wieder falsch werden zu lassen, muss ein neuer Termin größer als die aktuelle Zeit sein.
Der Informationsweg in der Abbildung beginnt bei diesem Vergleich und führt über die Freigaben zum Trap. MTIP, Machine Timer Interrupt Pending, ist die Anzeige der wartenden Timeranforderung in `mip`. MTIE, Machine Timer Interrupt Enable, ist die Timerkategorienfreigabe in `mie`. Hinzu kommt MIE, Machine

Interrupt Enable, die globale Freigabe in `mstatus`. Im Lehrmodell mit bereits laufendem Machine Mode müssen Pending, MTIE und MIE gemeinsam wirksam sein. Die Kästen stellen logische Bedingungen und keine zusätzlichen Zeitdauern dar. Der Vergleich erzeugt keine bloße kurze Flanke: Die fällige Bedingung besteht grundsätzlich weiter, bis sie beseitigt wird.
Die bekannte STM32-Konfiguration liefert einen nützlichen Vergleich. Im stetigen Aufwärtszählbetrieb mit unverändertem PSC, Prescaler oder Vorteiler, und ARR, Auto-Reload Register, durchläuft CNT, Counter, die Werte von null bis einschließlich ARR. Danach beginnt der Zähler wieder bei null. Die Periode umfasst damit ARR plus eins Zählschritte und entsteht unmittelbar in Hardware. Eine verspätete ISR, Interrupt Service Routine, verschiebt diese nächste Hardwaregrenze nicht.
Hundert Megahertz Timer-Eingangstakt, PSC neunundneunzig und ARR neunhundertneunundneunzig ergeben beispielsweise einen Megahertz Zählfrequenz und eine Millisekunde Periodendauer. Dieses Hardware-Raster driftet nicht durch verspätete ISR-Eintritte. Die Genauigkeit der Taktquelle oder eine geänderte Konfiguration bleiben eigene Einflüsse. Die Zahlen sind Lehrannahmen, keine Messwerte eines bestimmten STM32.
Der hier betrachtete RISC-V-Systemtimer setzt seine Zeitquelle dagegen nicht nach jeder gewünschten Periode zurück. `mtimecmp` legt einen einzelnen Vergleich fest; die Software muss ihn zum nächsten Termin weiterschieben. Funktional ähnelt er damit einem CCR, Capture/Compare Register, an einem frei laufenden Zähler. Die Analogie betrifft den geplanten Zeitpunkt, nicht identische Ereignis- und Quittierungsregeln. Ein RISC-V-System kann zusätzlich andere Peripherietimer mit eigener Periodenhardware enthalten.
Eine fortlaufende Zeitbasis ist besonders nützlich, wenn mehrere Nutzer Zeitdifferenzen bestimmen. Im klassischen Modell kann `mtime` mehreren Harts gemeinsam dienen, während jeder Hart einen zugeordneten Vergleich besitzt. Würde eine Anwendung die gemeinsame Zeitquelle zur Bildung ihrer eigenen Periode zurücksetzen, störte sie die Zeitrechnung der anderen Nutzer. Ein frei wählbarer nächster Termin ermöglicht außerdem tickless-Betrieb, also zeitgesteuertes Arbeiten ohne zwingenden periodischen Softwaretick. Anwendung oder Betriebssystem kann bis zum nächsten wirklich benötigten Termin warten. Weniger unnötige Aufweckvorgänge können bei passenden Schlafzuständen Energie sparen. Diese Eigenschaften erklären den Nutzen der Organisation, ohne eine vollständige historische Entwurfsbegründung zu behaupten. Zunächst bleibt ein periodisches Sollraster unser Anwendungsfall; dafür muss die Behandlung die fällige Hardwarebedingung korrekt zurücknehmen.

Folie 34 / 56

Rückkehr ist keine Timerquittierung

4 / 5 · Timer und Sollraster

Bei Eintritt gilt $\texttt{mtime} \geq \texttt{mtimecmp}$.

- `mret` stellt den Trap-Rückkehrzustand her
- Es setzt den Vergleichswert nicht automatisch weiter
- Der nächste Termin muss strikt in der Zukunft liegen

Was passiert bei Rückkehr mit unverändertem Vergleich und wirksamen Freigaben?

Notizen

Beim Eintritt einer Timerbehandlung ist die Bedingung `mtime` größer oder gleich `mtimecmp` erfüllt.

Eine Rückkehr ändert daran nichts. `mret`, Machine Return, stellt den Trap-Rückkehrzustand einschließlich der zuvor gesicherten globalen Interruptfreigabe wieder her, verändert aber den Vergleichswert nicht. Bei weiterhin fälligem Vergleich und wirksamen Freigaben kann deshalb unmittelbar eine weitere Timerbehandlung folgen.
Nur eine Softwarevariable zu erhöhen oder aus einer C-Funktion zurückzukehren beseitigt die Hardwareursache nicht. Die Behandlung schreibt stattdessen über den geeigneten Plattformzugriff einen neuen Vergleichstermin, der beim betrachteten Zeitstand strikt in der Zukunft liegt. Gleichheit genügt nicht, weil sie bereits als fällig zählt.
Auf realen Plattformen muss die Anzeige einer zurückgenommenen Timeranforderung eine Vergleichsänderung nicht verzögerungsfrei widerspiegeln. Ein robuster Ablauf verträgt deshalb auch einen zusätzlichen Eintritt, obwohl der nächste Solltermin noch nicht erreicht ist. Er liest die aktuelle Zeit und setzt nicht jeden Handler-Aufruf mit einer verstrichenen Periode gleich. Diese Trennung zwischen Hardwarezeit und Zahl der Behandlungseintritte wird in der Terminplanung wieder benötigt.
Beim bekannten STM32-Update-Betrieb liegt die Ursache anders. UIF, Update Interrupt Flag, speichert den Hinweis auf ein Update-Ereignis und muss nach den Regeln des Referenzhandbuchs behandelt und gelöscht werden. Bleibt es bei wirksamen Freigaben gesetzt, kann die Behandlung erneut angefordert werden. Beim RISC-V-Systemtimer ist dagegen der weiterhin wahre Vergleich die Ursache; ein Zukunftstermin nimmt diese Bedingung zurück. Unvollständige Ursachenbehandlung kann in beiden Fällen wiederholte Eintritte, einen Interruptsturm, erzeugen. Für das Hardware-Raster eines STM32-Update-Timers muss aber nicht in jeder ISR ein neues ARR geschrieben werden. Ein STM32-Compare-Kanal benötigt wiederum die Behandlung seines passenden Kanalflags, das vom Update-Flag zu unterscheiden ist. Vor dem ersten Eintritt müssen daher Vergleich, Freigaben und Behandlungspfad zueinander passend vorbereitet werden.

Folie 35 / 56

Erst vorbereiten, zuletzt freigeben

4 / 5 · Timer und Sollraster

1. Gültigen Stack und Trap-Eintritt sicherstellen
2. Globale Interruptfreigabe zunächst gesperrt halten
3. Zeit lesen, ersten Zukunftstermin nach `mtimecmp` schreiben
4. Timerkategorie freigeben: `mie.MTIE`
5. Globale Freigabe setzen: `mstatus.MIE`

Notizen

Die Initialisierung soll verhindern, dass eine bereits wartende Anforderung in unvorbereitete Software eintritt. Zuerst müssen deshalb Stack und Trap-Eintritt gültig sein. Der Stack nimmt die Sicherungsrahmen auf; sein Zeiger muss auf geeigneten Speicher zeigen. `mtvec`, Machine Trap-Vector Base Address, verweist auf einen passenden ersten Behandlungsbefehl. MIE, Machine Interrupt Enable in `mstatus`, bleibt bis zum Abschluss der Vorbereitungen gesperrt.
Danach liest die Software die Zeit konsistent und schreibt den ersten Zukunftstermin über eine geeignete Plattformfunktion nach `mtimecmp`, Machine Time Compare. Bei zehn Megahertz Zeitquellenfrequenz entspricht eine Millisekunde zehntausend Timer-Ticks. Ein möglicher erster Termin ist somit die gelesene Zeit plus zehntausend Schritte. Erst nach dieser Vorbereitung werden MTIE, Machine Timer Interrupt

Enable, in `mie` und zuletzt die globale Freigabe MIE in `mstatus` gesetzt. Nicht betroffene Registerfelder müssen dabei erhalten bleiben.
Die Bitpositionen dieser standardisierten Register sind in der privilegierten RISC-V-Spezifikation festgelegt: MIE liegt an Bit drei von `mstatus`, MTIE an Bit sieben und MEIE, Machine External Interrupt Enable, an Bit elf von `mie`. Die Anzeigen MTIP, Machine Timer Interrupt Pending, und MEIP, Machine External Interrupt Pending, stehen entsprechend an Bit sieben und elf von `mip`. Diese Positionen sind keine frei gewählten Plattformmerkmale. Welche Quellen und weiteren Funktionen tatsächlich vorhanden sind, bleibt dagegen eine Eigenschaft der Implementierung.
CSR steht für Control and Status Register, ein Steuer- und Statusregister im Prozessor. Die Geräteadressen von `mtime`, `mtimecmp` und PLIC gehören hingegen zu MMIO, Memory-Mapped Input/Output, dem Gerätezugriff über den Speicheradressraum. Ihre Adressen stammen wie die Timerfrequenz aus der Plattformbeschreibung. Eine standardisierte CSR-Bitposition ist daher von einer plattformabhängigen Geräteadresse und von der unterstützten Ausstattung zu unterscheiden.
Sind Stack, Trap-Eintritt und erster Vergleich vorbereitet, lassen sich die letzten beiden Freigabeschritte durch folgenden RISC-V-Assembler-Lehrausschnitt ausdrücken:

```
li    t0, 128        # Maske fuer MTIE: Bit 7
csrs  mie, t0        # Timerkategorie freigeben
csrsi mstatus, 8     # MIE: Bit 3, zuletzt freigeben
```

`li`, Load Immediate, lädt die Maske $2^7 = 128$ in das temporäre allgemeine Register `t0`. `csrs`, CSR Set, verknüpft diese Registermaske durch bitweises Oder mit dem bisherigen Inhalt von `mie`. `csrsi`, CSR Set Immediate, setzt MIE mit der unmittelbar angegebenen Maske acht. Beide Befehle erhalten die nicht ausgewählten Bits. Die unmittelbare Maske von `csrsi` ist nur fünf Bit breit: Acht passt hinein, hundertachtundzwanzig nicht. Für MTIE wird deshalb die Registerform `csrs` verwendet. Die verwandten Befehle `csrc` und `csrci`, CSR Clear beziehungsweise CSR Clear Immediate, löschen ausgewählte Bits anhand einer Register- oder unmittelbaren Maske.
Der Ausschnitt erklärt die Initialisierung und wurde nicht als RISC-V-Firmware ausgeführt. Ein vollständiges Bootprogramm benötigt außerdem passende Plattformzustände, Geräteordnung und definierte Fehlerpfade. Dauert die Vorbereitung länger als das gewählte Intervall, kann der erste Termin bei der Freigabe bereits fällig sein. Entscheidend ist dann, dass ein gültiger Handler bereitsteht. Für die folgenden periodischen Termine stellt sich darüber hinaus eine neue Frage: Soll ihre Folge vom tatsächlichen Bearbeitungsbeginn oder von den ursprünglich geplanten Zeitpunkten ausgehen?

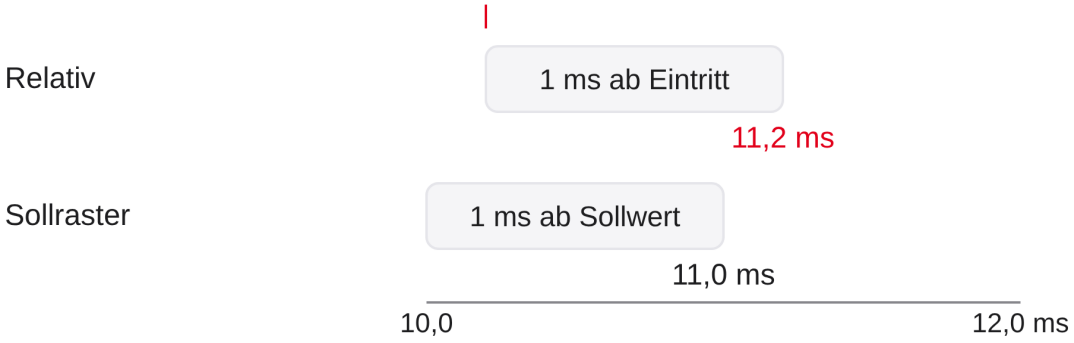
Folie 36 / 56

Vom ARR-Raster zum Softwaretermin

4 / 5 · Timer und Sollraster

STM32-Update: Hardware-Raster durch Prescaler und ARR.

Solltermin 10,0 ms · tatsächlicher Eintritt 10,2 ms



Brücke zum STM32-Compare: CNT läuft frei, ARR ist maximal.

- `CCR1 += period` bzw. $d + \Delta$: Sollraster
- `CCR1 = CNT + period` bzw. now $+ \Delta$: relativ

Notizen

Beim vertrauten periodischen STM32-Update-Betrieb wiederholt die Hardware die Periode selbst. PSC, Prescaler oder Vorteiler, bestimmt die Zählfrequenz; ARR, Auto-Reload Register, begrenzt die Periode. CNT, Counter, durchläuft null bis ARR und beginnt wieder bei null. Solange diese Einstellungen unverändert bleiben, verschiebt eine verspätete ISR, Interrupt Service Routine, die folgenden Hardwaregrenzen nicht. Das Sollraster besteht bereits unabhängig vom Ausführungszeitpunkt der Behandlung.

Der RISC-V-Systemtimer wiederholt eine gewünschte Softwareperiode dagegen nicht durch Zurücksetzen seiner Zeitquelle. `mtime`, Machine Time, zählt weiter, während `mtimecmp`, Machine Time Compare, nur den nächsten Vergleichstermin enthält. Die Software muss deshalb selbst festlegen, von welchem Zeitpunkt sie den folgenden Termin berechnet. Die beiden Bahnen der Abbildung zeigen genau diese Planungsentscheidung, nicht das automatische Wiederholen durch ARR.

Eine engere Brücke bietet der STM32-Output-Compare-Betrieb, also ein Betrieb mit Vergleichsereignissen. Ein geeigneter Timer lässt sich mit ARR am maximalen Zählwert frei laufen. CCR1, Capture/Compare Register des ersten Kanals, enthält einen Vergleichstermin. Nach einem Treffer kann Software diesen Vergleich weiterstellen. Die schematischen Alternativen `CCR1 += period` und `CCR1 = CNT + period` entsprechen dem Fortschieben des bisherigen Solltermins beziehungsweise einer neuen Wartezeit ab der aktuell gelesenen Zeit. `period` ist dabei eine positive Periode in Zählschritten des jeweiligen Timers. Diese Registerprinzipien sind kein vollständiger STM32-Treiber.

Mehrere verfügbare Compare-Kanäle können auf diese Weise eigene Terminfolgen auf demselben Timer erzeugen. Sie teilen sich CNT und den Vorteiler; unabhängig sind ihre Vergleichstermine, nicht ihre Taktquellen. Ein Kanal kann etwa einen Ein-Millisekunden-Termin verfolgen und ein anderer ein anderes Raster. Welche Kanäle und Betriebsarten vorhanden sind, legt der konkrete Timer fest.

Die Analogie hat Grenzen. Auch ein STM32-Zähler mit maximalem ARR läuft irgendwann um; eine Übertragung auf reale Rohzähler benötigt daher eine passende Umlaufbehandlung. Außerdem entsteht der hier betrachtete STM32-Compare-Treffer bei Gleichheit von CNT und CCR1 und setzt CC1IF, Capture/Compare

1 Interrupt Flag. Wird ein bereits verpasster Vergleich eingetragen, kann der nächste Treffer erst nach dem Zählerumlauf auftreten. Der RISC-V-Vergleich bleibt dagegen anstehend, solange `mtime` mindestens `mtimecmp` erreicht. Ein geeigneter Zukunftstermin und die richtige Ursachenbehandlung sind in beiden Systemen nötig, aber ihre Mechanismen sind nicht identisch.

Für die Planungsentscheidung nehmen wir nun eine gewünschte Periode von einer Millisekunde an. Der vorherige Solltermin war $10{,}0$ ms, der Handler beginnt jedoch erst bei $10{,}2$ ms, etwa weil zuvor noch eine andere nicht verschachtelte Behandlung lief. Muss der nächste Termin nun bei $11{,}2$ ms oder bei $11{,}0$ ms liegen? Beide Antworten können sinnvoll sein, sie erfüllen aber unterschiedliche Anforderungen. In der Abbildung bezeichnet Rot die Verspätung. `now` ist die aktuell gelesene Zeit, d der bisherige Solltermin und Δ die positive Periode. Alle Größen müssen dieselbe Einheit besitzen. Die Zeitachse verwendet Millisekunden, die Software die entsprechend umgerechneten Timer-Ticks.

	Relativ	Absolut, Sollraster
Formel	now $+ \Delta$	$d + \Delta$
Nächster Termin im Beispiel	$11{,}2$ ms	$11{,}0$ ms
Wirkung der Verspätung	wird in die Phase folgender Solltermine übernommen	verschiebt das Sollraster nicht
Passt für	neue Wartezeit ab einem gewählten Ereignis oder Aktionsende	gleichmäßiges Raster von Sollterminen

Die relative Planung geht hier vom tatsächlichen Handler-Eintritt aus: $10{,}2 + 1 = 11{,}2$ ms. Sie übernimmt die Verspätung in die folgende Terminfolge. Die absolute Planung verwendet dagegen den geplanten Vorgänger: $10 + 1 = 11$ ms. Sie hält damit das Sollraster ein, entsprechend dem Fortschieben des bisherigen STM32-CCR-Werts. Eine andauernde Blockierung kann auch die nächste tatsächliche Behandlung verspäten; eine andere Rechnung beseitigt diese Blockierung nicht. Die numerische Periode in Ticks hängt zudem von der Zählfrequenz ab. Eine Millisekunde umfasst bei einem Megahertz tausend, bei zehn Megahertz zehntausend Schritte.

Soll nach Abschluss einer Aktion ein Mindestabstand verstreichen, muss die relative Planung tatsächlich deren Abschluss als Bezugspunkt verwenden. Ein beim Handler-Eintritt gelesener `now`-Wert gewährleistet nicht denselben Abstand nach einer erst anschließend beendeten längeren Aktion. Der Bezugspunkt gehört deshalb zur zeitlichen Anforderung.

Wiederholen sich Eintrittsverspätungen bei relativer Planung, kann die Terminfolge zunehmend vom ursprünglichen Raster abwandern. Dieser fortlaufende Phasenversatz heißt hier Drift. Als zusätzliches Lehrbeispiel seien eine konstante Eintrittsverspätung von fünf Mikrosekunden je Periode und keine weiteren Verzögerungen angenommen. Das ist kein gemessener oder allgemein typischer Latenzwert. Bei einer Sollperiode von tausend Mikrosekunden liegen die tatsächlichen Eintritte dann $1000 + 5 = 1005\,\mu$s auseinander. Das Intervall ist $0{,}5$ Prozent länger als gewünscht.

Eine Softwareuhr, die bei jedem Eintritt trotzdem nur eine Millisekunde addiert, läuft im Verhältnis $1000/1005$ zur realen Zeit. Nach einer realen Stunde verliert sie ungefähr

$$3600\,\text{s} \cdot \left(1 - \frac{1000}{1005}\right) \approx 17{,}91\,\text{s} \approx 18\,\text{s}.$$

Die Hardwarezeit `mtime` läuft dadurch nicht langsamer. Der Fehler entsteht aus relativer Neuplanung und dem Zählen von Handler-Aufrufen. Verstrichene Zeit wird deshalb zuverlässiger aus einer geeigneten Zeitquelle gewonnen.

Absolute Planung erhält die Solltermine, verhindert aber nicht alle Schwankungen der tatsächlichen Behandlung. Solche zeitlichen Schwankungen heißen Jitter. Bei exakt gleicher Verspätung von fünf Mikrosekunden wären sämtliche Eintritte nur gemeinsam verschoben, ohne diese Schwankung. Bei wechselnder Verspätung bleibt Jitter auch bei absoluter Planung bestehen. Ihr langfristiger mittlerer Abstand nähert sich der Sollperiode nur unter passenden Voraussetzungen, insbesondere wenn alle Termine behandelt werden und die Verzögerung begrenzt bleibt. Die Genauigkeit der Hardware-Zeitquelle ist zusätzlich zu berücksichtigen. Ein eingehaltenes Sollraster garantiert daher keine exakt gleichmäßigen tatsächlichen Reaktionen.

Ein Sollraster passt zu regelmäßigen Verarbeitungs- und Anzeigeterminen, zur Signalabtastung oder zu einem digitalen Regler mit fester Abtastzeit. Viele Abtast- und Reglermodelle setzen gleichmäßige Zeitabstände voraus. Ein Software-Sollraster allein stellt den tatsächlichen Abtastzeitpunkt jedoch nicht sicher. Ist dieser entscheidend, kann eine timerabhängige Hardwareerfassung erforderlich sein, während die Auswertung später erfolgen darf. Auch eine Uhr benötigt eine geeignete Zeitquelle; regelmäßige Anzeigeaufrufe sind davon zu unterscheiden.

Eine relative Wartezeit passt dagegen zu einer neuen Frist ab einem bestimmten Ereignis: Tasterentprellung zwanzig Millisekunden nach der letzten erfassten Flanke, ein Empfangs-Timeout nach dem letzten Byte oder die Erholzeit eines Sensors nach einer abgeschlossenen Messung. Bei einer neuen relevanten Flanke oder einem neuen Byte wird die jeweilige Frist erneut von diesem Ereignis aus berechnet.

Anschaulich lässt sich die Unterscheidung am Pflanzengießen machen. „Täglich um acht Uhr gießen“ beschreibt ein Sollraster. „Vierundzwanzig Stunden nach dem letzten Gießen“ beschreibt eine relative Wartezeit. Wird einmal erst um zehn Uhr gegossen und tritt danach keine weitere Verspätung auf, plant das Sollraster den nächsten Morgen wieder um acht Uhr, die relative Planung dagegen um zehn Uhr. Beide Regeln sind korrekt, wenn sie zur jeweiligen Absicht passen.

Liegt die Verspätung bei mindestens einer ganzen Periode, genügt auch das einfache Fortschieben um $d + \Delta$ nicht mehr. Dieser neue Termin ist dann bereits fällig, weil er vor oder genau auf `now` liegt. Für einen zurückgenommenen Timervergleich wird der erste Rasterpunkt strikt in der Zukunft benötigt. Dafür muss die Software bestimmen, wie viele Solltermine inzwischen verstrichen sind. Ein Systemtick ist somit ein Softwarekonzept wiederkehrender Termine und weder ein Nachweis rechtzeitiger Ausführung noch ein zwingendes Merkmal eines Betriebssystems.

Folie 37 / 56

Ein Flag zählt keine Perioden

4 / 5 · Timer und Sollraster

STM32: 6 Update-Ereignisse können in `UIF = 1` zusammenfallen.

RISC-V-Beispiel: $\Delta = 1$ ms, nächster Solltermin $d = 10$ ms, aktuelle Zeit $n = 15{,}3$ ms.

$$k = \left\lfloor \frac{n - d}{\Delta} \right\rfloor + 1 = 6, \qquad d' = d + k\Delta = 16 \text{ ms}$$

Ein Handler-Aufruf bedeutet hier sechs fällige Solltermine.

Nachzählen braucht eine erhaltene Zeitbasis, nicht nur ein Flag.

Notizen

Ein regelmäßiges Hardware-Raster und die Zahl ausgeführter Interruptbehandlungen sind unterschiedliche Größen. Das gilt bereits für den bekannten STM32-Update-Betrieb: Die Hardwareperioden laufen auch bei gesperrter Interruptbehandlung weiter. Das erste durch den Zählerüberlauf erzeugte Update-Ereignis setzt UIF, Update Interrupt Flag, auf eins. Weitere Ereignisse setzen dasselbe bereits gesetzte Bit erneut. Nach sechs Perioden sieht die Software deshalb möglicherweise immer noch nur ein Flag. Erhöht eine später einmal ausgeführte ISR, Interrupt Service Routine, ihre Softwarezeit um genau eine Periode, fehlen fünf Perioden. Das Hardware-Raster wurde nicht verschoben; die Information geht beim bloßen Zählen der ISR-Aufrufe verloren.

UIF und der periodisch zurückgesetzte CNT, Counter, reichen ohne weitere Zeitinformation im Allgemeinen nicht aus, um die Zahl vergangener voller Perioden zu rekonstruieren. Mit einem zusätzlichen frei laufenden Zähler und einer passenden Umlaufregel ist dies auch auf einem STM32 möglich. Im verwendeten RISC-V-Modell liefert `mtime`, Machine Time, eine solche fortlaufende Zeit bereits. Der Unterschied ist daher eine Eigenschaft der gewählten Zeitorganisation, keine grundsätzliche Grenze aller STM32-Timer.

Aus dieser Zeit lässt sich die Zahl fälliger Termine berechnen. d bezeichnet den bisher nächsten Solltermin, n die aktuell gelesene Zeit und Δ die positive Periode. Alle drei Größen müssen dieselbe Einheit verwenden. Die anschauliche Rechnung verwendet Millisekunden, die spätere Implementierung Timer-Ticks. Zählerumlauf und arithmetischer Überlauf sind für den betrachteten Zeitraum ausgeschlossen.

Bei $d = 10$ ms, $n = 15{,}3$ ms und $\Delta = 1$ ms sind die Termine zehn, elf, zwölf, dreizehn, vierzehn und fünfzehn Millisekunden fällig. Die Abrundung $\lfloor (15{,}3 - 10)/1 \rfloor = 5$ zählt die zusätzlichen Perioden seit d. Zusammen mit dem ursprünglichen Termin ergeben sich

$$k = \left\lfloor \frac{n - d}{\Delta} \right\rfloor + 1 = 6, \qquad d' = d + k\Delta = 16 \text{ ms}.$$

d' liegt strikt in der Zukunft und weiterhin auf demselben Sollraster. Nur einmal auf elf Millisekunden zu erhöhen, ließe den Vergleich dagegen fällig und könnte nach der Rückkehr sofort weitere Interrupts erzeugen.
Die sechs Termine werden aus der Zeitdifferenz erschlossen, nicht aus einem Pending-Bit oder einer einzelnen Wartemarkierung. Ob die Anwendung sechs logische Aktionen nachholt, mehrere Termine zusammenfasst oder Überlast meldet, bleibt eine weitere Entscheidung. Ein einzelner Handler-Aufruf kann vergangene Zeit berücksichtigen, ohne dass dadurch bereits die zu jedem Termin gewünschten Daten vorhanden wären.

Folie 38 / 56

Sechs Termine – sechsmal messen?

4 / 5 · Timer und Sollraster

Im zuvor betrachteten Beispiel „Ein Flag zählt keine Perioden“ sind die Termine 10 bis 15 ms fällig. Der Handler läuft erst bei $15{,}3$ ms.

Ziel der Anwendung	Mögliche Reaktion
Verstrichene Zeit bestimmen	Zeitdifferenz aus dem Hardwarezähler
Eine aktuelle Anzeige erneuern	einmal mit aktuellem Zustand auswerten
Jeden historischen Messwert erhalten	zuvor gespeicherte Daten benötigen
Regelmäßig Messwerte erfassen	Hardware-Trigger und ausreichenden Puffer prüfen

Ein Sollraster ersetzt keine gespeicherten Messwerte.

Notizen

Sechs fällige Solltermine erklären eine verstrichene Zeit und den nächsten passenden Vergleich. Sie liefern jedoch keine nachträglichen Messungen. Wenn die Behandlung erst bei $15{,}3$ ms beginnt, können sechs unmittelbar nacheinander ausgeführte Sensorlesungen die Messwerte der vergangenen Zeitpunkte zehn bis fünfzehn Millisekunden nicht rekonstruieren. Ein Handler ist eine Behandlungsroutine, kein Speicher früherer physikalischer Zustände.
Auch ein korrektes STM32-Hardware-Raster mit ARR, Auto-Reload Register, ändert daran nichts. UIF, Update Interrupt Flag, erhält nur den Hinweis auf mindestens ein Update, weder eine vollständige Ereignisanzahl noch einzelne Messwerte. Umgekehrt speichert `mtime`, Machine Time, zwar eine fortlaufende Zeitbasis, aber ebenfalls keine Sensordaten. Eine ISR, Interrupt Service Routine, kann die Zahl verstrichener Termine berechnen, verlorene Beobachtungen jedoch nicht nachträglich herstellen.
Die passende Reaktion hängt deshalb vom Anwendungsziel ab. Für eine Anzeige kann es genügen, einmal den aktuellen Zustand auszuwerten und überholte Zwischenstände zu überspringen. Eine Zeitberechnung verwendet die Differenz zur Hardware-Zeitquelle. Wer jeden historischen Messwert erhalten muss, benötigt dagegen eine rechtzeitige Erfassung und genügend Speicher bis zur Übernahme. Daten, die zuvor nicht aufgezeichnet wurden, lassen sich durch späteres Lesen nicht wiederherstellen.
Ein Hardware-Trigger ist ein auslösendes Ereignis aus Hardware. Soweit die Plattform es unterstützt, kann

ein Timer damit eine Messwerterfassung zum geplanten Zeitpunkt anstoßen, während deren Auswertung später stattfindet. Auch dann muss ein ausreichend großer Puffer die erfassten Werte bewahren. Ein gleichmäßiges Raster, ein Interruptflag und ein Datenpuffer erfüllen drei getrennte Aufgaben: Zeitpunkt vorgeben, auf Arbeit hinweisen und Daten erhalten.
Damit kehrt das Problem zum Messgerät vom Einstieg zurück. Die Zahl fälliger Termine und die Zahl tatsächlich verfügbarer Messwerte dürfen nicht gleichgesetzt werden. Nachholen, Zusammenfassen oder eine Überlastreaktion muss aus den Anforderungen begründet werden; ungeprüftes Nachholen kann die Verzögerung sogar vergrößern. Die anschließende Hilfsfunktion beschränkt sich deshalb auf den zukünftigen Vergleichstermin und entscheidet bewusst nicht über die fachliche Nachholpolitik.

Folie 39 / 56

Nächsten Vergleichstermin planen

4 / 5 · Timer und Sollraster

C-Lehrmodell: monotone Zeit, positive Periode, kein Umlauf oder Überlauf.

```c
uint64_t next_future(uint64_t d, uint64_t now,
                     uint64_t period) {
    if (now < d)
        return d;
    uint64_t k = (now - d) / period + 1u;
    return d + k * period;
}
```

Ergebnis nach `mtimecmp`; beim STM32-CCR zusätzlich den Zählerumlauf behandeln.

Notizen

Die Terminrechnung lässt sich vom Gerätezugriff trennen. Die C-Funktion `next_future` erhält `d` als bisherigen nächsten Solltermin, `now` als aktuelle Zeit und `period` als positive Periode. Alle Angaben stehen in Timer-Ticks. `uint64_t` aus `stdint.h` ist ein vorzeichenloser Vierundsechzig-Bit-Typ; `k` zählt die fälligen Termine. Vorausgesetzt werden monotone Zeit sowie kein Zählerumlauf oder arithmetischer Überlauf im betrachteten Zeitraum. Eine Periode null ist nicht zulässig, weil durch `period` geteilt wird.
Liegt `now` noch vor `d`, ist der bisherige Termin weiterhin zukünftig und wird unverändert zurückgegeben. Andernfalls ermittelt die ganzzahlige Division die Zahl vollständig verstrichener Intervalle seit `d`. Für positive Werte entspricht sie der mathematischen Abrundung. Durch das anschließende Addieren von eins wird auch der bereits fällige Termin `d` berücksichtigt. Der zurückgegebene Wert liegt damit strikt hinter `now` und bleibt auf dem ursprünglichen Raster.
Bei zehntausend Ticks pro Millisekunde ergeben `d=100000`, `now=153000` und `period=10000` den Zukunftstermin `160000`. Für `now=99999` bleibt `100000` bestehen; bei genau `100000` geht es bereits auf `110000` weiter. Der Gleichheitsfall ist wichtig, weil der Timervergleich einen erreichten Termin bereits als fällig behandelt.
Eine ISR, Interrupt Service Routine, kann zunächst `mtime`, Machine Time, konsistent lesen und das Ergebnis der Rechnung über eine geeignete Plattformfunktion nach `mtimecmp`, Machine Time Compare,

schreiben. Ein zusätzlicher Eintritt bei noch zukünftigem Termin wird dabei nicht irrtümlich als verstrichene Periode gezählt. Die Hilfsfunktion selbst quittiert jedoch kein Gerät und enthält weder Trap-Prolog noch Kontextwiederherstellung. Auch die konsistente Weitergabe eines mehrteiligen Zeit-Snapshots an andere Programmteile bleibt ein eigener Schritt. Die reine Rechnung kann auf dem Host geprüft werden, ohne damit eine ausgeführte RISC-V-Firmware zu behaupten.
Funktional entspricht das Fortschieben von `mtimecmp` dem Verschieben eines CCR, Capture/Compare Register, an einem frei laufenden STM32-Timer. Im rechtzeitig bedienten Ein-Perioden-Fall genügt der bisherige Vergleich plus Periode. Bei mehreren fälligen Terminen springt die Funktion bis zum ersten zukünftigen Rasterpunkt. Wie viele fachliche Aktionen nachgeholt werden sollen, entscheidet sie weiterhin nicht.
Für eine konkrete STM32-Übertragung ist die Annahme der nicht umlaufenden Zeit besonders zu beachten. CNT, Counter, bleibt auch bei maximalem ARR, Auto-Reload Register, ein endlicher Zähler. Ein Rohwert wird nicht dadurch zu einer fortlaufenden Zeitbasis, dass er nach `uint64_t` umgewandelt wird. Notwendig sind eine passende Umlaufbehandlung oder eine korrekt erweiterte Zeitbasis sowie eine Festlegung der zulässigen Abstände zwischen Vergleichsterminen. Der C-Lehrcode ist daher keine unverändert einsetzbare STM32-Treibervorlage.
Außerdem muss der neue Vergleich rechtzeitig wirksam werden. Bei der betrachteten STM32-Gleichheitsprüfung kann ein bereits verpasster CCR-Termin bis zum nächsten Zählerumlauf auf seinen Treffer warten; das passende Kanalflag ist zusätzlich zu quittieren. Beim RISC-V-Vergleich bleibt ein fälliger Termin dagegen anstehend, bis ihn ein geeigneter Zukunftswert ablöst. Nach einer langen Behandlung ist in beiden Fällen zu prüfen, wie viel Zeit bis zum neu programmierten Termin tatsächlich noch verbleibt.
Nicht jede Anwendung benötigt überhaupt eine Periode. Im tickless-Betrieb kann die Software direkt den nächsten relevanten absoluten Zeitpunkt auswählen. `next_future` löst speziell das Fortschieben eines periodischen Sollrasters und setzt deshalb eine positive Periode voraus. Die Hardware-Zeitquelle wird in keinem dieser Fälle für die Anwendungsperiode zurückgesetzt. Nachdem externe und zeitgesteuerte Ursachen damit korrekt behandelt werden können, bleibt die gemeinsame Systemfrage offen: Wie schnell sind die Daten gesichert, und wie gelangen sie zuverlässig zur späteren Auswertung?

Folie 40 / 56

5 · Rechtzeitig und konsistent reagieren

Wann sind die Daten wirklich gesichert – und wann ist die Arbeit fertig?
Ereignisse → Trap-Eintritt → Geräte → Timer → Reaktionszeit
Kurze ISR, vollständiges Zeitbudget und sichere Übergabe zusammendenken.

Notizen

Ereignisspeicherung, Trap-Pfad, Gerätebehandlung und Timerplanung sind nun als einzelne Bausteine bekannt. Für einen funktionierenden Systementwurf müssen sie zusammenpassen. Eine korrekte Unterbrechungsbehandlung kann trotzdem zu spät beginnen; ein rechtzeitig gesetzter Hinweis kann bei seiner Übergabe verloren gehen. Der fünfte Abschnitt verbindet deshalb zeitliche Anforderungen mit der Bedeutung und Konsistenz der gespeicherten Daten.
Zuerst wird festgelegt, welcher Teil der Arbeit sofort nötig ist und welcher später im Hauptprogramm erfolgen darf. Danach wird der gesamte Weg vom Ereignis bis zum verlangten Endpunkt berechnet. Die Sicherung eines Messwerts und der Abschluss seiner Auswertung können sehr unterschiedliche Fristen

erfüllen. Der mittlere Aufwand einer kurzen ISR, Interrupt Service Routine, genügt dafür allein nicht als Nachweis.
Die Aufteilung erzeugt schließlich gemeinsam genutzten Zustand. An einem Flag wird sichtbar, warum beobachtbare einzelne Zugriffe noch keine unteilbare Übernahme-und-Lösch-Folge ergeben. Mit diesen Erkenntnissen lässt sich die Frage des Messgeräts erneut beantworten: Die rechtzeitige Sicherung benötigt einen begrenzten Reaktionspfad; die spätere Auswertung benötigt eine konsistente Übergabe und auf Dauer genügend Verarbeitungskapazität.

Folie 41 / 56

Kurze Behandlung, spätere Nutzarbeit

5 / 5 · Reaktionszeit und Übergabe

Ein Hart: ISR und Hauptprogramm laufen nacheinander.



- ISR: erforderliche zeitkritische Schritte
- Hauptprogramm: längere Verarbeitung
- Eine Markierung allein ist nicht schon die fertige Reaktion

Notizen

Ein korrekt vorbereiteter Timer- oder Gerätepfad kann eine Unterbrechungsbehandlung auslösen. Damit ist noch nicht entschieden, welche Arbeit dort ausgeführt werden soll. Die entscheidende Trennung verläuft zwischen dem sofort erforderlichen Geräteschritt und der später möglichen Auswertung. Eine ISR, Interrupt Service Routine, sichert beispielsweise einen bedrohten Messwert, behandelt die Geräteursache und veröffentlicht die Information, dass weitere Arbeit vorliegt. Eine längere Filterrechnung kann anschließend das Hauptprogramm übernehmen.
Die Zeitachse stellt diese Aufteilung dar: Das Hauptprogramm wird unterbrochen, die kurze Behandlung läuft, danach setzt das Hauptprogramm seine Arbeit fort und bearbeitet die gesicherten Daten. Foreground bezeichnet hier die vorrangige ISR, Background das zwischen den Interrupts laufende Hauptprogramm. Beide teilen sich denselben Hart, also denselben Hardware-Ausführungskontext. Sie laufen zeitlich verschränkt und nicht gleichzeitig auf zwei Kernen. Deferred Handling bedeutet entsprechend aufgeschobene Behandlung.
Die Aufteilung verkürzt die Zeit, während der andere Interrupts im nicht verschachtelten Grundmodell warten müssen. Sie verlagert jedoch einen Teil der Reaktion in die Zukunft. Zwischen der Veröffentlichung eines Hinweises und seiner Auswertung kann erneut Wartezeit liegen. Ein gesetztes Flag oder ein Zeit-Snapshot belegt daher noch keine fertige Ausgabe. Für einen vollständigen Entwurf müssen sowohl der Aufwand der ISR als auch die Verzögerung bis zur Nutzarbeit und die Kapazität der Datenübergabe bekannt sein.

Folie 42 / 56

Kurze ISR: Aufwand berechnen

5 / 5 · Reaktionszeit und Übergabe
Lehrwerte: Prozessor 100 MHz, 1000 Behandlungen pro Sekunde, 500 Prozessorzyklen je vollständiger ISR.

$$t_{\text{ISR}} = \frac{500}{100 \cdot 10^6} = 5\,\mu\text{s}$$

$$U = 1000\,\text{s}^{-1} \cdot 5\,\mu\text{s} = 0{,}5\,\%$$

Beweist dieser kleine Zeitanteil eine Reaktion binnen $100\,\mu$s?

Notizen

Wie viel Prozessorzeit eine Unterbrechungsbehandlung beansprucht, lässt sich im vorgegebenen Lehrmodell aus ihrer Zykluszahl und dem Prozessortakt bestimmen. Die fünfhundert Prozessorzyklen umfassen hier die gesamte ISR, Interrupt Service Routine, einschließlich Ein- und Austritt. Bei hundert Megahertz führt die Rechnung

$$t_{\text{ISR}} = \frac{500}{100 \cdot 10^6\,\text{s}^{-1}} = 5\,\mu\text{s}$$

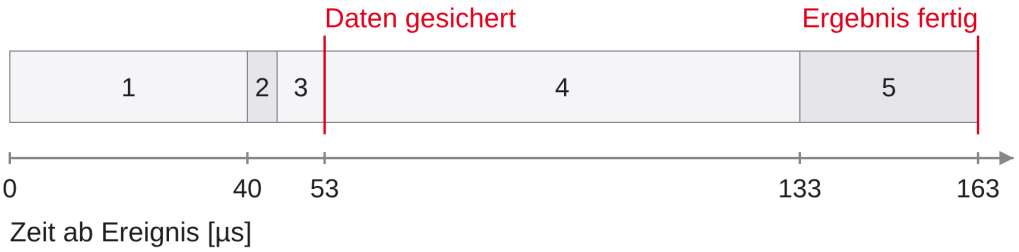
zur Dauer einer vollständigen Behandlung. Die angenommene Zykluszahl ist eine Vorgabe dieses Modells, kein aus C-Quellcode abgeleiteter allgemeiner Laufzeitwert.
Bei tausend solchen Behandlungen pro Sekunde werden insgesamt fünf Millisekunden Prozessorzeit je Sekunde benötigt. Der mittlere Anteil beträgt damit $U = 0{,}005 = 0{,}5\,\%$. Vorausgesetzt ist, dass tatsächlich tausend vollständige Behandlungen dieses Aufwands stattfinden. Ob die Hardware tausend Einzelereignisse unterscheidbar gespeichert hat, ist eine andere Frage: Ein einziges Pending-Bit kann mehrere Ereignisse weiterhin zusammenfassen.
Der kleine Zeitanteil beweist keine Reaktion innerhalb von hundert Mikrosekunden. Ein einzelnes Ereignis könnte während einer langen Interruptsperre eintreffen oder auf das Ende einer anderen Behandlung warten. Wird seine Auswertung aufgeschoben, kommt zusätzliche Wartezeit im Hauptprogramm hinzu. Auslastung beschreibt den mittleren Ressourcenbedarf; eine Frist verlangt dagegen eine obere Grenze für den gesamten Weg bis zu einem genau benannten Endpunkt.

Folie 43 / 56

Wann ist die Reaktion fertig?

5 / 5 · Reaktionszeit und Übergabe

Zwei Endpunkte derselben Ereignisbehandlung



1 Blockierung: 40 µs 2 Eintritt: 5 µs 3 Sicherung: 8 µs

4 Warten auf Hauptprogramm: 80 µs 5 Auswertung: 30 µs

- Daten gesichert: $40 + 5 + 8 = 53\,\mu$s
- Ergebnis fertig: $53 + 80 + 30 = 163\,\mu$s

Eine $100\,\mu$s-Frist erfüllt nur der erste Endpunkt.

Notizen

Eine Reaktionszeit beginnt beim betrachteten Ereignis und endet bei der geforderten Wirkung. Im Messgerät können mindestens zwei verschiedene Wirkungen gemeint sein: Der neue Messwert ist gegen Überschreiben gesichert, oder seine Auswertung ist vollständig abgeschlossen. Diese Endpunkte dürfen nicht gleichgesetzt werden.

Die Zeitachse setzt das Ereignis auf den Anfang ihrer Mikrosekundenskala. Im Lehrmodell dauert die Blockierung durch gesperrte Interrupts oder eine bereits laufende, nicht verschachtelte Behandlung höchstens vierzig Mikrosekunden. Bis zum Beginn der eigenen Gerätearbeit kommen höchstens fünf Mikrosekunden Eintrittslatenz hinzu. Acht weitere Mikrosekunden genügen zur Datensicherung. Der erste rote Endpunkt liegt deshalb bei

$$t_{\text{gesichert}} = 40 + 5 + 8 = 53\,\mu\text{s}.$$

Die spätere Auswertung wartet im Hauptprogramm höchstens weitere achtzig Mikrosekunden und benötigt anschließend dreißig Mikrosekunden. Ihr Abschluss liegt bei $t_{\text{fertig}} = 53 + 80 + 30 = 163\,\mu$s. Die Segmentbreiten entsprechen den jeweiligen Dauern; die beiden roten Marken gehören zum selben Ereignis.

Die Forderung, den Wert binnen hundert Mikrosekunden in den Speicher zu übernehmen, ist unter diesen Annahmen erfüllt. Eine ebenso kurze Frist bis zum ausgewerteten Ergebnis wäre verletzt. Alle Dauern sind hier angenommene Obergrenzen ohne zusätzliche, nicht eingerechnete Last. Für eine reale Zusage im ungünstigsten zulässigen Fall wäre eine Analyse der konkreten Plattform und aller relevanten Belastungen erforderlich. Die Rechnung erläutert das Reaktionsbudget und stellt keine Messung einer Hardwarelatenz dar.

Folie 44 / 56

Was bleibt in der ISR?

5 / 5 · Reaktionszeit und Übergabe

Sofort nötiger Schritt	Häufig aufschiebbare Arbeit
Daten vor Überschreiben sichern	längere Filterrechnung
Geräteursache passend quittieren	Text formatieren
Nächsten Timervergleich setzen	Bedienoberfläche aktualisieren
Übernommene Quelle abschließen	gespeicherte Daten weiter auswerten

Die Aufteilung folgt Frist, Gerätevertrag und Speicherbedarf.

Notizen

Das Reaktionsbudget bestimmt, welche Arbeit unmittelbar erledigt werden muss. Ein Messwert, der nach hundert Mikrosekunden überschrieben wird, benötigt zunächst eine rechtzeitige Übernahme in einen geeigneten Speicher. Danach kann seine längere Auswertung warten, sofern deren eigene Anforderungen dies erlauben. Die Grenze zwischen sofortiger und späterer Arbeit folgt somit den Fristen, der Gerätebeschreibung und dem verfügbaren Speicher.

Zu den unmittelbar notwendigen Schritten gehören häufig das Sichern der Daten und das Behandeln der Geräteursache. Bei einer externen Quelle muss außerdem die übernommene Controlleranforderung vollständig abgeschlossen werden. Beim Timer wird der nächste Vergleichstermin vorbereitet. Die rechte Tabellenspalte nennt dagegen Arbeit, die sich oft verschieben lässt: Filterrechnung, Textformatierung, Aktualisierung einer Bedienoberfläche oder weitere Auswertung bereits gesicherter Daten. Diese Zuordnung ist keine starre Regel für jedes Gerät.

Im ursprünglichen Messgerät kann die vier Millisekunden dauernde Rechnung unterbrechbar im Hauptprogramm bleiben, während eine kurze ISR, Interrupt Service Routine, die neue Messung sichert. Müsste bereits das vollständige Auswertungsergebnis binnen hundert Mikrosekunden vorliegen, würde diese Aufteilung allein die Anforderung nicht erfüllen.

Aufgeschobene Arbeit benötigt zudem ein ausreichendes Zeit- und Speicherbudget. Kommen dauerhaft mehr Daten an, als verarbeitet werden können, ist irgendwann jeder endliche Puffer voll. Schon bevor dieser Durchsatz betrachtet wird, entsteht eine weitere Korrektheitsfrage: Wie übergibt die ISR ihren Hinweis oder ihre Daten so an das Hauptprogramm, dass kein widersprüchlicher oder verlorener Zustand entsteht?

Folie 45 / 56

Gemeinsam genutzter Zustand

5 / 5 · Reaktionszeit und Übergabe

- ISR und Hauptprogramm greifen zeitlich verschränkt zu
- Snapshot: veröffentlichter Beobachtungsstand
- Ein Bit kann Ereignisse zusammenfassen
- Mehrschrittige Änderungen benötigen ein Konsistenzverfahren

Notizen

Die Aufteilung in unmittelbare Behandlung und spätere Auswertung verbindet zwei Programmteile durch gemeinsam genutzten Zustand. Obwohl nur ein Hart arbeitet, kann die ISR, Interrupt Service Routine, das Hauptprogramm zwischen zwei seiner Schritte unterbrechen. Diese zeitliche Verschränkung genügt, um ungeschützte mehrschrittige Zugriffe fehlerhaft werden zu lassen.
Ein Snapshot ist ein veröffentlichter Beobachtungsstand, beispielsweise eine aus der Hardwarezeit abgeleitete Anzahl vergangener Millisekunden. Er darf bis zur nächsten Veröffentlichung veralten, soll aber einen in sich stimmigen Wert enthalten. Im festgelegten Plattformmodell kann ein natürlich ausgerichtetes Zweiunddreißig-Bit-Wort durch einen unteilbaren Zugriff gelesen oder geschrieben werden. Das ist eine Annahme über Hardware und verwendete C-Implementierung; sie entsteht nicht allein durch das Schlüsselwort `volatile`.
Ein Vierundsechzig-Bit-Snapshot kann auf RV32 mehrere Zugriffe benötigen. RV32 besitzt zweiunddreißig Bit breite allgemeine Integerregister. Unterbricht eine Aktualisierung das Lesen der beiden Hälften, kann ein Mischwert entstehen. Selbst bei einem unteilbar gelesenen Wort bleibt eine Folge aus Lesen, Entscheiden und Zurückschreiben eine mehrschrittige Gesamtaktion.
Eine kurze kritische Sektion kann die betreffenden Interrupts für die Übernahme sperren. Anschließend stellt sie den zuvor geltenden Freigabestand wieder her. Blindes Einschalten wäre falsch, wenn der aufrufende Code die Interrupts bereits absichtlich gesperrt hatte. Andere Lösungen verwenden passend entworfene atomare oder versionierte Datenstrukturen. Welche Lösung nötig ist, richtet sich nach der Bedeutung und Breite der übergebenen Information.

Folie 46 / 56

`volatile` verhindert keinen verlorenen Hinweis

5 / 5 · Reaktionszeit und Übergabe

Schritt	Hauptprogramm	ISR
1	liest Flag 1	
2		setzt erneut Flag 1
3	schreibt Flag 0	

Die zweite Markierung ist danach nicht mehr sichtbar.

Notizen

Ein einzelnes Flag kann die Aussage „mindestens eine Bearbeitung angefordert“ festhalten. Die Tabelle beginnt mit einem bereits gesetzten Flag: Das Hauptprogramm liest den Wert eins und will die vorgemerkte Arbeit übernehmen. Bevor es das Flag löscht, tritt eine weitere ISR, Interrupt Service Routine, ein und schreibt erneut eins. Danach setzt das Hauptprogramm das Flag auf null. Der währenddessen neu entstandene Hinweis ist anschließend nicht mehr sichtbar.
Das Problem liegt in der Reihenfolge der Zugriffe. Beide einzelnen Schreibzugriffe können unteilbar sein, ohne dass die Folge aus Lesen und Löschen insgesamt unteilbar wäre. `volatile` verlangt die entsprechenden beobachtbaren Zugriffe nach den Regeln der verwendeten C-Implementierung. Es verbindet sie weder zu einer atomaren Gesamtaktion noch stellt es eine Warteschlange bereit. Die Tabellenzeilen beschreiben nacheinander ausgeführte Schritte auf einem Hart, keine gleichzeitig arbeitenden Prozessoren.
Zwei Entscheidungen müssen getrennt werden. Zunächst ist festzulegen, ob mehrere Ereignisse überhaupt zusammengefasst werden dürfen. Für eine erneute Anzeige des aktuellen Zustands kann ein Bit genügen; für drei getrennt benötigte Messwerte genügt es nicht. Anschließend ist das Übernahmeverfahren zu bestimmen. Selbst bei bewusst zusammengefassten Anforderungen darf ein neu gesetztes Flag nicht durch eine ältere Übernahme gelöscht werden. Eine kurze, gegen die betreffende ISR geschützte Übernahme-und-Lösch-Aktion löst diesen zweiten Fehler, ohne die längere Nutzarbeit in den gesperrten Bereich zu verlagern.

Folie 47 / 56

Übernahme und Löschen zusammen schützen

5 / 5 · Reaktionszeit und Übergabe

Pseudocode: ein Hart, ein Arbeitsflag, keine verschachtelte ISR.

```
vorherige Freigabe merken und relevante IRQ sperren
    lokal = arbeitsflag
    arbeitsflag = 0
vorherige Freigabe wiederherstellen
wenn lokal gesetzt: längere Arbeit erledigen
```

Die geschützte Folge bleibt kurz; die Nutzarbeit liegt außerhalb.

Notizen

Die kritische Sektion verbindet zwei Schritte, zwischen denen kein neuer Hinweis verloren gehen darf: Das Hauptprogramm übernimmt das Arbeitsflag in eine lokale Kopie und setzt den gemeinsamen Zustand auf null. Während dieser kurzen Folge kann die betreffende ISR, Interrupt Service Routine, nicht eintreten. Nach dem Wiederherstellen der Freigabe bleibt ein neuer Hinweis für die nächste Verarbeitungsrunde stehen.

IRQ bedeutet Interrupt Request, Unterbrechungsanforderung. Der Pseudocode benennt keine Funktionen einer konkreten Bibliothek, sondern deren erforderliche Wirkung. Die Plattform muss das Merken des Freigabestands, die Interruptsperre, ihre Wiederherstellung und die benötigte Compilerordnung gewährleisten. `lokal` wird nur vom Hauptprogramm verwendet und benötigt deshalb während der anschließenden Auswertung keine gemeinsame Aktualisierung.

Bereits vor der Übernahme zusammengefallene Anforderungen bleiben bei diesem Verfahren zusammengefasst. Das ist mit der vereinbarten Flagbedeutung „mindestens eine Bearbeitung angefordert“ verträglich. Soll jede Anforderung gezählt oder jeder Messwert erhalten werden, ist zusätzlich ein geeigneter Zähler oder Datenpuffer erforderlich. Zugriffskonsistenz und Informationsgehalt lösen unterschiedliche Probleme.

Die längere Arbeit erfolgt nach dem Freigeben, damit sie andere Reaktionen nicht blockiert. Wiederhergestellt wird genau der vorherige Freigabestand; eine schon zuvor gewollte Sperre darf nicht verloren gehen. Auf mehreren Harts schützt diese lokale Interruptsperre nicht vor einem anderen Kern. Der gezeigte Ablauf ist deshalb ausdrücklich auf das einzelne, nicht verschachtelt arbeitende Hart-Modell begrenzt.

Folie 48 / 56

Flag, Zähler oder Datenpuffer?

5 / 5 · Reaktionszeit und Übergabe

Benötigte Aussage	Geeignete Grundidee
Aktuellen Zustand erneut auswerten	Flag, Ereignisse dürfen zusammenfallen
Wie viele Ereignisse sind eingetroffen?	konsistent übernommener Zähler
Welche einzelnen Messwerte kamen an?	ausreichend großer Datenpuffer

Auch Zähler und Puffer brauchen Überlauf- und Übergaberegeln.

Notizen

Die Wahl der Übergabestruktur richtet sich danach, welche Information die Anwendung später noch benötigt. Ein Flag hält nur eine Zustandsaussage fest. Es passt beispielsweise zur Aufforderung, die Anzeige des aktuellen Zustands neu aufzubauen: Mehrere solche Aufforderungen dürfen zu einer einzigen offenen Arbeit zusammenfallen. Wie viele Ereignisse sie ausgelöst haben, bleibt unbekannt.
Ein Zähler bewahrt die Anzahl der Ereignisse. Seine Aktualisierung und Übernahme müssen konsistent sein, und ein Überlauf darf nicht unbeachtet bleiben. Die einzelnen Messwerte sind dadurch noch nicht gespeichert. Wer auch diese Werte benötigt, braucht einen Datenpuffer, gegebenenfalls mit Zeitstempeln. Dieser erhält Daten nur innerhalb seiner Kapazität und bei einer eindeutigen Vereinbarung darüber, welcher Programmteil welchen Eintrag gerade beschreiben oder lesen darf.
Ein Ringpuffer verwendet seine Speicherplätze zyklisch wieder. Seine vollständige Implementierung ist hier noch nicht erforderlich; zunächst muss begründet werden, ob Anzahl, einzelne Werte oder lediglich eine offene Bearbeitungsanforderung gebraucht werden. Die spätere Vorlesung zur Datenbewegung vertieft die Verwaltung solcher Puffer.
Keine dieser Eigenschaften folgt aus `volatile`. Das Schlüsselwort beeinflusst beobachtbare Zugriffe gemäß der verwendeten C-Implementierung, ergänzt aber weder Speicherplätze noch eine atomare mehrschrittige Übergabe. Atomar bedeutet unteilbar im jeweils benötigten Zugriffsmodell. Ein vollständiger Entwurf verbindet deshalb die passende Information mit einem konsistenten Übergabeverfahren und einer Regel für den Fall, dass die verfügbare Kapazität nicht genügt.

Folie 49 / 56

Zurück zum Messgerät

5 / 5 · Reaktionszeit und Übergabe

Neue Daten binnen $100\ \mu$s sichern, Auswertung dauert 4 ms.

- Rechengeschwindigkeit allein begrenzt die Abfragelücke nicht passend
- Hardware muss Daten und Bereitschaft bis zur Übernahme erhalten
- Kurze ISR kann sichern und quittieren; Auswertung folgt später
- Frist und Übergabe brauchen eigene Begründungen

Was ändert dauerhafter Messbetrieb?

Notizen

Der Ausgangsfall lässt sich nun mit einem vollständigen Ereignis- und Zeitpfad beantworten. Ein Mikrocontroller wertet vier Millisekunden lang alte Messdaten aus; sein ADC, Analog-to-Digital Converter oder Analog-Digital-Umsetzer, liefert währenddessen einen neuen Wert, der nach hundert Mikrosekunden überschrieben wird. Den Prozessor idealisiert doppelt so schnell zu machen, verkürzt die Rechnung nur auf zwei Millisekunden. Eine Abfrage erst danach verfehlt die Frist weiterhin.

Polling ist trotzdem grundsätzlich möglich, wenn seine größte tatsächliche Abfragelücke einschließlich Datenübernahme ausreichend klein bleibt. Ein Interrupt eröffnet einen anderen Ausführungspfad. Auch dessen Blockierung, Eintrittslatenz und Datensicherung müssen begrenzt sein. Die begründete Antwort lautet daher nicht, dass Interrupts unter allen Bedingungen überlegen wären, sondern dass der gewählte Weg die konkrete Frist nachweislich erfüllen muss.

Im betrachteten Entwurf hält das Gerät den Wert bis zur Übernahme fest. Die kurze ISR, Interrupt Service Routine, sichert ihn, behandelt die Ursache und schließt die Controlleranforderung ab. Eine konsistente Übergabe ermöglicht die spätere Auswertung. Mit den angenommenen Zeitgrenzen liegt der Wert nach dreiundfünfzig Mikrosekunden im Speicher; das ausgewertete Ergebnis folgt erst nach hundertdreiundsechzig Mikrosekunden. Die ursprüngliche Sicherungsfrist ist damit erfüllt, eine gleich kurze Ergebnisfrist wäre es nicht.

Für dauerhaften Messbetrieb kommt die Durchsatzfrage hinzu. Übersteigt die mittlere Ankunftsrate unbegrenzt die Verarbeitungskapazität, wächst die wartende Datenmenge trotz korrekt eingehaltenem Einzelzugriff. Kein endlicher Puffer beseitigt diesen Unterschied. Weniger Daten, kürzere Auswertung, zulässiges Zusammenfassen oder Hardwareunterstützung können den Entwurf verändern. Kurze Lastspitzen lassen sich dagegen gegebenenfalls puffern. Rechtzeitige Übernahme, korrekte Übergabe und ausreichender Durchsatz bleiben drei getrennt zu prüfende Anforderungen.

Abschlussquiz: Welche Annahme fehlt?

5 / 5 · Reaktionszeit und Übergabe

- Zehn Fragen, zehn Minuten beantworten, fünf Minuten auswerten
- Prüfen Sie Ereignisspeicherung, Freigaben und vollständige Behandlung
- Unterscheiden Sie Zeitquelle, Solltermin und Handler-Aufruf
- Begründen Sie strittige Antworten mit Zustand oder Zeitrechnung

Die optionalen Vertiefungen werden nicht vorausgesetzt.

Notizen

Die zehn Quizfragen verdichten die wesentlichen Unterscheidungen des Hauptteils. Jede Frage besitzt genau eine richtige Antwort; die kurze Erläuterung begründet, welche Modellannahme oder welcher Zustand den Ausschlag gibt. Die anschließenden optionalen Implementierungsdetails werden dafür nicht benötigt.

Der erste Themenbereich betrifft die Ereignisspeicherung und die Freigaben des Trap-Pfads. Ein gespeichertes Bereitschaftsbit bewahrt nicht automatisch mehrere Daten, und eine wartende Anforderung bedeutet noch keine erlaubte Unterbrechung. Beim Kontext sind die von Hardware gesicherten Spezialregister von den zusätzlich durch Software zu erhaltenden allgemeinen Registern zu unterscheiden. Im Gerätepfad beantworten Interruptkategorie und Geräte-ID unterschiedliche Fragen; Claim, Geräteaktion und Complete erfüllen unterschiedliche Aufgaben. Auch eine höhere Quellenpriorität hebt die fehlende Verschachtelung des Grundmodells nicht auf.

Die Timerfragen trennen Frequenz und Einheit der Zeitquelle vom Prozessortakt sowie den Solltermin vom Zeitpunkt eines Handler-Aufrufs. Die abschließenden Entscheidungen betreffen mittlere Auslastung, einzelne Reaktionsfristen und konsistente Flagübergabe. Ein kleiner mittlerer Zeitanteil ist kein Nachweis einer kurzen maximalen Wartezeit, und `volatile` macht eine mehrschrittige Übergabe nicht unteilbar. Eine strittige Antwort lässt sich durch eine konkrete Zustandsfolge oder Zeitrechnung prüfen. Beispielsweise widerlegt ein verspäteter Handler-Aufruf die Gleichsetzung von Aufrufzahl und verstrichenen Sollperioden. Das Quiz ergänzt dadurch die offenen Papieraufgaben: Es macht einzelne Fehlvorstellungen schnell erkennbar, während dort ein vollständiger eigener Lösungsweg hergeleitet wird.

Was Sie jetzt begründen können

5 / 5 · Reaktionszeit und Übergabe

- Ein Ereignis braucht Erfassung, Benachrichtigung und Behandlung
- Freigaben und Kontext machen einen Trap-Pfad korrekt
- Claim, Geräteaktion und Complete haben verschiedene Aufgaben
- Zeitquelle und absolutes Sollraster begrenzen Planungsfehler
- Kurze ISR und konsistente Übergabe brauchen ein Reaktionsbudget

Vorlesung 4 nutzt diese Grundlage für zeitgesteuerte Ausgänge.

Notizen

Die fünf Abschnitte ergeben einen zusammenhängenden Weg vom physikalischen Ereignis bis zur nutzbaren Information. Zuerst muss das Ereignis einen hinreichenden gespeicherten Zustand hinterlassen. Polling oder Interrupt benachrichtigen die Software darüber; keiner der beiden Wege ersetzt die benötigte Datenspeicherung. Für den betrachteten Machine Mode bestimmen die Freigaben, ob eine wartende Interruptanforderung einen Trap auslösen darf.

Beim Eintritt erhält die Hardware den speziellen Trap-Zustand. Software sichert zusätzlich die benötigten allgemeinen Register und ermöglicht damit eine korrekte Fortsetzung. Bei externen Geräten übernimmt der PLIC, Platform-Level Interrupt Controller, die Quellenauswahl. Claim übernimmt eine Quelle, die Geräteaktion beseitigt deren tatsächliche Ursache, und Complete gibt die weitere Weiterleitung dieser Quelle wieder frei. Diese Schritte dürfen nicht miteinander verwechselt werden.

Für Zeitaufgaben liefert `mtime`, Machine Time, die Zeitbasis; `mtimecmp`, Machine Time Compare, legt den nächsten Vergleichstermin fest. Ein absolutes Sollraster verhindert, dass jede Bearbeitungsverspätung dauerhaft in die Phase der folgenden Termine eingeht. Es garantiert jedoch noch keine rechtzeitige Ausführung und stellt keine verpassten Messwerte wieder her. Erst ein vollständiges Reaktionsbudget und eine konsistente Übergabe verbinden Terminplanung und Anwendung.

Die Grundbegriffe orientieren sich an Harris/Harris, Digital Design and Computer Architecture — RISC-V Edition. Architekturdetails sind in der RISC-V Machine-Level-Spezifikation, der Gerätecontroller in der PLIC-Spezifikation beschrieben. Die folgende Vertiefung erläutert vier ausgewählte Implementierungsfragen und ist keine Voraussetzung für Übung oder Quiz. Die nächste Vorlesung verwendet Zeitquelle, Hardwarevergleich und begründete Reaktion für Ausgänge und Überwachung.

Folie 52 / 56

Vertiefung · Zugriffe und Sonderfälle

Wie erfüllen konkrete Zugriffe die Verträge des Hauptteils?
Selbstlektüre · nicht vorausgesetzt für Übung und Quiz
RV32-Halbzugriffe · Verschachtelung · PLIC-Schwellenwert

Notizen

Der Hauptteil verwendet bewusst klare Schnittstellenverträge: Die Zeit wird konsistent gelesen, der Vergleich korrekt geschrieben und eine Behandlung ohne weitere Unterbrechung ausgeführt. Diese Verträge genügen, um Ereigniswege und Reaktionszeiten zu begründen. Ihre Umsetzung erfordert bei einer konkreten Plattform zusätzliche Entscheidungen.
Die folgenden vier Vertiefungen erläutern ausgewählte Grenzen dieser Vereinfachungen. Auf RV32 müssen zweiunddreißig Bit breite Zugriffe zu einem stimmigen vierundsechzig Bit breiten Zeitwert verbunden werden. Beim Schreiben eines Vergleichstermins dürfen Zwischenwerte keine künstlich verfrühte Anforderung erzeugen. Wird eine laufende Behandlung erneut unterbrochen, müssen mehrere Rückkehrzustände außerhalb der einzelnen Trap-Register gesichert werden. Beim PLIC unterscheiden sich schließlich die Bedingungen für eine Benachrichtigung von denen eines ausdrücklich ausgeführten Claim-Zugriffs.
Diese Beispiele erweitern das Verständnis der bereits verwendeten Modelle. Sie sind weder neue Voraussetzungen für die Pflichtaufgaben noch vollständige Treiberimplementierungen. Insbesondere ersetzen sie keine Plattformbeschreibung, keine Prüfung der Gerätezugriffsordnung und keinen Nachweis eines realen Zeitverhaltens.

Folie 53 / 56

Vertiefung: `mtime` auf RV32 lesen

Optionale Vertiefung
C-Lehrfunktion; `mt[0]` ist LO, `mt[1]` HI:

```
uint64_t read_mtime32(volatile uint32_t *mt) {
    uint32_t hi, lo, hi_again;
    do {
        hi = mt[1];
        lo = mt[0];
        hi_again = mt[1];
    } while (hi != hi_again);
    return ((uint64_t)hi << 32) | lo;
}
```

Notizen

Der Hauptteil setzt einen konsistent gelesenen Hardwarezeitwert voraus. Auf RV32 ist dieser Vertrag nicht schon durch einen einzelnen normalen Wortzugriff erfüllt: `mtime` ist vierundsechzig Bit breit, während die allgemeinen Integerregister zweiunddreißig Bit umfassen. Die obere und untere Registerhälfte müssen deshalb passend zusammengefügt werden, obwohl der Hardwarezähler zwischen den Zugriffen weiterläuft. Die C-Lehrfunktion verwendet die Typdefinitionen aus `stdint.h`. `uint32_t` und `uint64_t` sind vorzeichenlose Ganzzahltypen mit genau zweiunddreißig beziehungsweise vierundsechzig Bit, soweit die Implementierung sie bereitstellt. `mt` zeigt auf die untere Hälfte des angenommenen Registers; die nächste Wortposition enthält die obere Hälfte. Vorausgesetzt sind Little Endian, hier die untere Hälfte an der kleineren Adresse, passend ausgerichtete Wortzugriffe und die angegebene Gerätezugriffsreihenfolge. Die Funktion liest zuerst HI in `hi`, dann LO in `lo` und anschließend HI erneut in `hi_again`. Unterschiedliche obere Werte zeigen einen Übertrag während des Versuchs an; die Schleife wiederholt. Sind sie gleich, lässt sich die untere Hälfte dem passenden oberen Bereich zuordnen, sofern der Versuch nicht einen ganzen Umlauf der unteren Hälfte umfasst. Vor der Linksverschiebung um zweiunddreißig Bit wird nach `uint64_t` erweitert; das bitweise Oder fügt LO hinzu.
Ein Beispiel verdeutlicht den Fehler eines ungeprüften Halbzugriffs: HI gleich eins vor und LO gleich null nach dem Übergang zu `0x00000002_00000000` ergäben gemeinsam einen falschen Mischwert. Das zweite HI gleich zwei erkennt den Übergang. Eine Interruptsperre würde den Hardwarezähler nicht anhalten. `volatile` verlangt beobachtbare Zugriffe gemäß der C-Implementierung, ersetzt aber keine beliebige plattformspezifische Speicherordnung. Die Funktion erläutert daher einen Zugriff unter klaren Annahmen; Geräteadresse und tatsächlich geeignete Zugriffsfunktion stammen weiterhin von der konkreten Plattform.

Folie 54 / 56

Vertiefung: `mtimecmp` auf RV32 schreiben

Optionale Vertiefung

C-Lehrfunktion für Little Endian und stark geordnete Gerätezugriffe:

```
void write_compare32(volatile uint32_t *cmp,
                     uint64_t value) {
    cmp[0] = UINT32_MAX;
    cmp[1] = (uint32_t)(value >> 32);
    cmp[0] = (uint32_t)value;
}
```

Ein einzelner `uint64_t`-Store garantiert auf RV32 keine atomare Aktualisierung.

Notizen

Beim Schreiben des vierundsechzig Bit breiten Vergleichstermins tritt das Gegenstück zum Leseproblem auf. Zwei getrennte Wortschreibungen können vorübergehend einen Mischwert erzeugen, der kleiner als sowohl der alte als auch der neue Termin ist. Die Timerbedingung könnte dadurch künstlich zu früh wahr werden. Ein `uint64_t`-Store im C-Quelltext garantiert auf RV32 keine atomare Geräteaktualisierung. Die gezeigte Dreischrittfolge verwendet Little Endian und stark geordnete Gerätezugriffe. `cmp` zeigt auf

die untere Hälfte von `mtimecmp`, Machine Time Compare, und `value` enthält den neuen Gesamtwert. Die Typen `uint32_t` und `uint64_t` stammen aus `stdint.h`; `UINT32_MAX` ist der Wert `0xFFFFFFFF` des vorzeichenlosen Zweiunddreißig-Bit-Typs.
Zuerst wird die untere Hälfte auf ihren maximalen Wert gesetzt. Danach folgt die obere Hälfte des gewünschten Termins; zuletzt erhält die untere Hälfte ihren endgültigen Wert. Der erste Zwischenwert ist nicht kleiner als der alte Termin, der zweite nicht kleiner als der neue. Damit entsteht kein Zwischenvergleich unterhalb des kleineren der beiden Gesamtwerte. Die Folge verhindert diesen künstlichen Zusatzfehler; sie verschiebt einen absichtlich schon fälligen Endtermin nicht in die Zukunft.
Die Zugriffsreihenfolge muss tatsächlich in der Gerätelogik wirksam werden. `volatile` allein begründet diese Hardwareeigenschaft nicht. Ein zweiter gleichzeitig schreibender Handler ist im Modell ausgeschlossen, und die konkrete Registeradresse ist eine Plattforminformation. Die Funktion ist damit ein begrenztes Lehrmodell für den Vergleichszugriff, kein vollständiger Timer- oder Trap-Treiber.

Folie 55 / 56

Vertiefung: Verschachtelte Interrupts

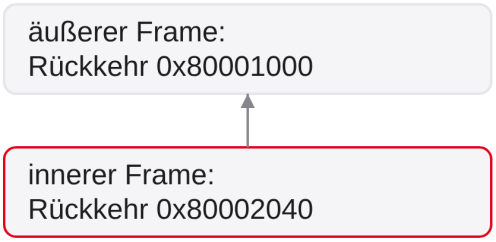
Optionale Vertiefung

Ein Hardware-mepc, getrennte Softwareframes bei Nesting

äußerer Eintritt: mepc 0x80001000

innerer Eintritt: mepc 0x80002040

Ohne Sicherung ist die ursprüngliche Adresse verloren.



Rückkehr: innen → außen → Main; Register und Status ebenfalls erhalten.

Ein `mepc` ist kein Stapel von Rückkehradressen.
Vor Wiederfreigabe: Kontext sichern, Quellen und Stackbedarf begrenzen.

Notizen

Das Grundmodell verhindert weitere maskierbare Interrupts während einer laufenden ISR, Interrupt Service Routine. Die Hardware setzt beim Trap-Eintritt das Bit MIE, Machine Interrupt Enable, auf null; die Software lässt es dort. Wird es innerhalb der Behandlung wieder gesetzt, kann eine weitere freigegebene Anforderung eintreten. Diese Verschachtelung benötigt zusätzliche Sicherung, weil die speziellen Trap-Register keinen Stapel früherer Zustände bilden.
Das Beispiel beginnt mit einem äußeren Trap, der das Hauptprogramm bei `0x80001000` unterbricht. Die Adresse steht zunächst in `mepc`, Machine Exception Program Counter. Ein innerer Trap bei `0x80002040` innerhalb der äußeren Behandlung ersetzt den Registerinhalt durch diese neue Fortsetzungsadresse. Ohne eigene Software-Sicherung ist die ursprüngliche Hauptprogrammadresse verloren. Die innere Rückkehr kann dann korrekt sein, während die spätere äußere Rückkehr an die falsche Stelle führt.
Getrennte Sicherungsrahmen im Stack erhalten die verschachtelten Zustände. Die rechte Bildhälfte zeigt diese Softwareframes; die Pfeile kennzeichnen die umgekehrte Reihenfolge der Rückkehr. Zu sichern sind

auch die benötigten allgemeinen Register, der Status und geänderte Masken, nicht nur zwei Adresswerte. Der Platzbedarf des Stacks und die Menge erneut zugelassener Quellen müssen begrenzt sein.
Die erforderliche Sicherung muss vor der Wiederfreigabe existieren. Die Darstellung begründet dieses Prinzip, liefert aber keine bereits geprüfte sichere Assemblerimplementierung. Für die Entscheidungen des Hauptteils bleibt deshalb die nicht verschachtelte Behandlung maßgeblich: Eine höhere Quellenpriorität wählt die nächste Behandlung aus und unterbricht die laufende nicht automatisch.

Folie 56 / 56

Vertiefung: Was der PLIC-Threshold filtert

Optionale Vertiefung
Für die Benachrichtigung eines Kontexts:

$$\text{pending} \wedge \text{enabled} \wedge (\text{priority} > \text{threshold})$$

- Priorität 0: Quelle unwirksam
- Bei gleicher aktiver Priorität gewinnt die kleinere Geräte-ID
- Expliziter Claim und Benachrichtigung sind getrennte Vorgänge

Notizen

Im Hauptteil bleibt der Schwellenwert des PLIC-Kontexts auf null. Ein höherer Threshold erlaubt eine zusätzliche Unterscheidung zwischen der Benachrichtigung eines Empfängers und dessen ausdrücklicher Übernahme einer Quelle. PLIC bedeutet Platform-Level Interrupt Controller; der Kontext ist der Empfänger aus Hart und Privilegstufe.
Für eine Benachrichtigung muss eine Quelle wartend, für den Kontext freigegeben und mit einer Priorität strikt oberhalb seines Thresholds versehen sein. Pending bezeichnet den wartenden Zustand, Enable die Quellenfreigabe und Priority ihre Rangordnung. Bei Threshold fünf löst eine Quelle mit Priorität fünf noch keine Benachrichtigung aus; Priorität sieben genügt unter den übrigen Voraussetzungen. Priorität null macht die Quelle unwirksam. Bei gleicher positiver Priorität entscheidet die kleinere Geräte-ID, also ihre Identifikationsnummer.
Die Controllerbenachrichtigung allein führt noch nicht zum Trap. Im Hart müssen außerdem die externe Kategorienfreigabe und die globale Freigabe passen. Der Threshold verändert weder die Verdrahtung und IDs noch die eingestellten Quellenprioritäten.
Ein expliziter Claim ist von dieser Benachrichtigungsbedingung getrennt und unabhängig vom Threshold. Eine wartende, freigegebene Quelle mit Priorität drei kann bei Threshold fünf zwar keine Benachrichtigung erzeugen, bei einem bewusst ausgeführten Claim aber dennoch übernommen werden. Die Funktionen „Benachrichtigung unterdrücken“, „Rangordnung bestimmen“ und „eine wartende Quelle übernehmen“ sind deshalb nicht austauschbar. Dieser Sonderfall ergänzt die Grundentscheidung aus dem PLIC-Abschnitt; er ist für die Pflichtaufgaben mit Schwellenwert null nicht erforderlich.